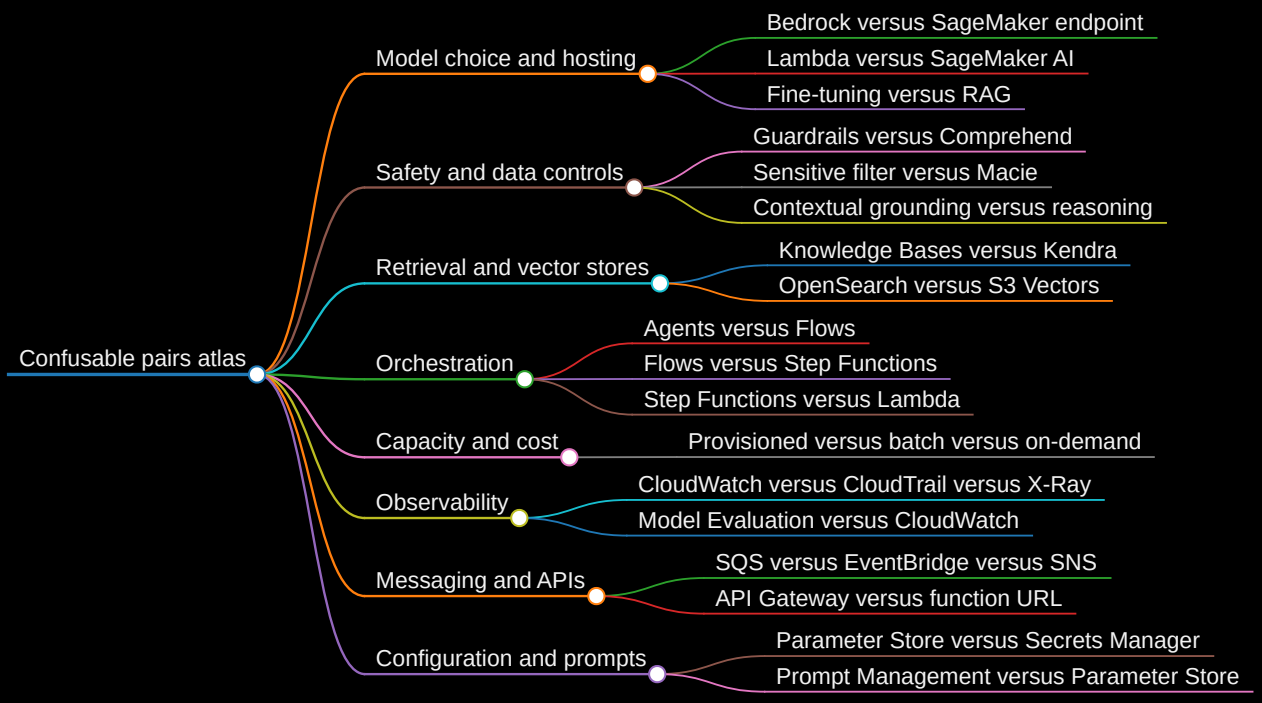
Confusable pairs atlas: how to tell look-alike answers apart in long questions

After this module, you can read a long scenario, find the one constraint that decides the service, and reject each look-alike option with one reason. Most lost points in this exam come from two services that do part of the same job. The fix is to ask what the service does at run time, what it accepts as input, and what it returns. This module teaches that test for 32 pairs, ordered by how often they appear in the practice corpus.



The deciding constraint

Most long questions contain five to eight services, and only one or two of them match the stated constraint. Start with the constraint word, not with the service names. Words about who owns the runtime, how long the job runs, who must read the output, and when a rule applies decide most answers. The tree below starts from a scenario

and ends at a service family. Use it first, then open the pair section for the two services that remain.

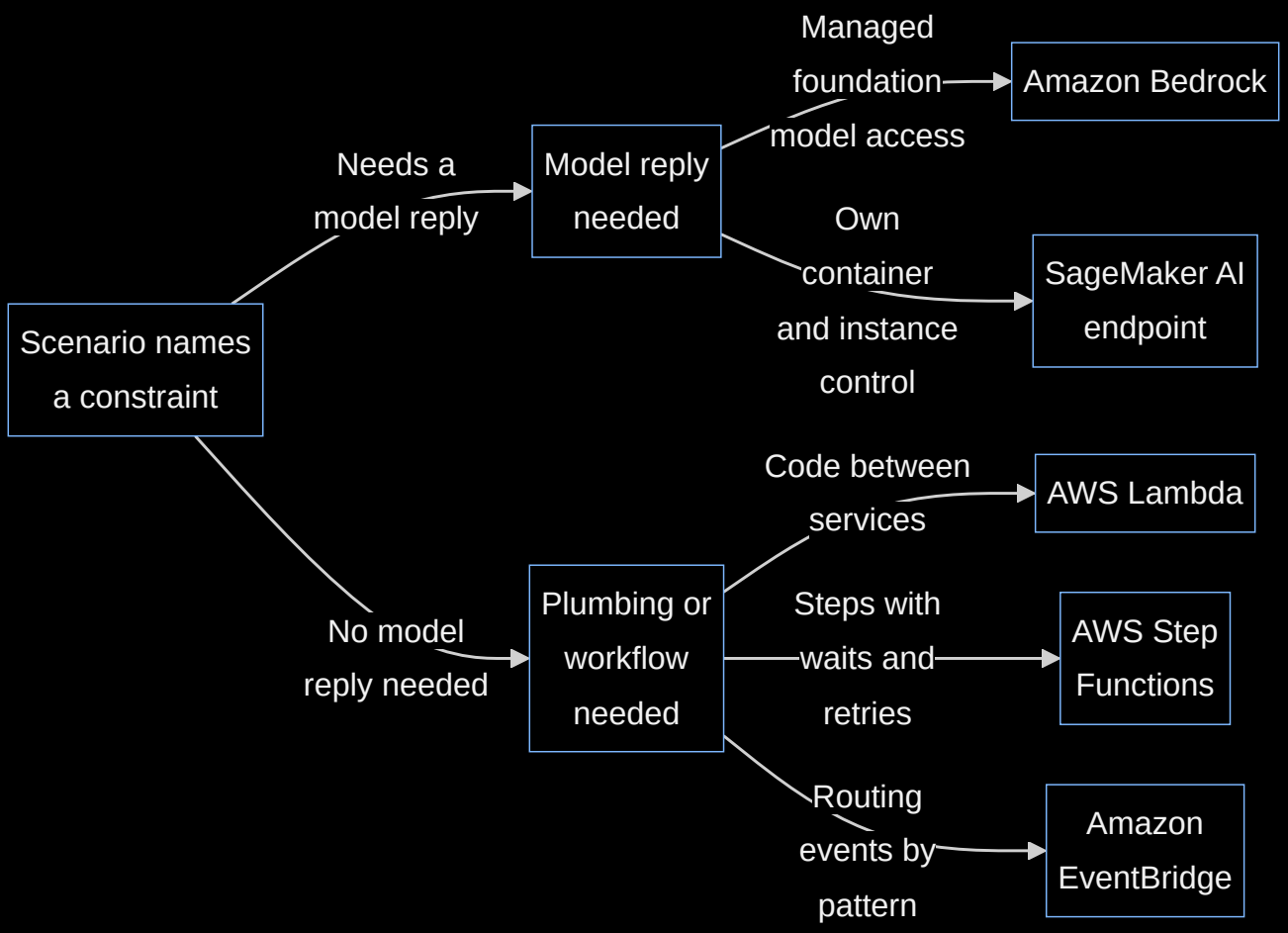


Figure: First split the scenario by whether a model reply is required.

Signal words that point to a model reply: generate, summarize, draft, answer, foundation model, token, prompt, grounded. **Signal words that point to plumbing:** trigger, retry, wait, route, queue, schedule, event, handler. The same sentence can contain both, so decide what the question asks you to choose.

Rule: Name the one constraint in the scenario before you read any option. Write it in your own words.

Exam tip: Long stems often add a sentence that changes the answer, such as a limit on data location, a latency target, or a staff role. Underline that sentence first.

Confusable pairs atlas

Each section below gives one pair. The count line shows how often the practice corpus put the two services against each other. Counts add both directions, and labels that differ only by the words Amazon Bedrock are merged. A count of 0 means the two services did not meet directly in the corpus. In those cases, the nearest row is shown. The pairs with a count of 0 are still in the atlas because the learner needs them.

Each pair uses the same layout. You get a decision tree, three lines for purpose, input, and effect, one question to ask yourself, the signal words for each side, and the trap. The source line lists the AWS pages or Sundog slides behind the facts.

Bedrock versus Lambda

Corpus count: 98 (74 and 24 in the two directions). The most frequent pair in the corpus.

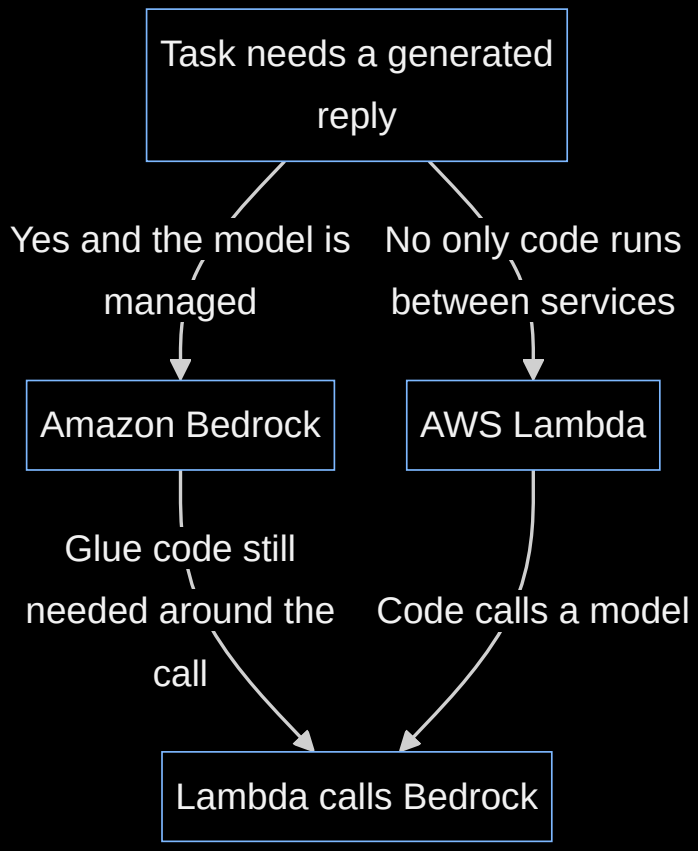


Figure: Bedrock returns the reply. Lambda decides when and how the call happens.

- **Purpose:** Bedrock gives managed access to foundation models. Lambda runs your code in response to an event.
- **Input:** Bedrock takes messages or a prompt plus model settings. Lambda takes an event from a trigger, such as an API call or a queue message.
- **Effect:** Bedrock returns generated content. Lambda returns whatever your code returns, often after calling Bedrock.

Ask yourself: Does the requirement describe the content of a reply, or the code that runs around the reply?

- **Bedrock signal words:** model, foundation model, prompt, token, summarize, grounded answer, guardrail.
- **Lambda signal words:** event, trigger, handler, transform payload, call an API, retry in code, glue.

Trap: Lambda appears in most correct architectures, so it looks like the answer whenever it is named. Lambda is the caller in these designs. Pick it only when the requirement is about code between services.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html, Sundog slide page 118.

Bedrock versus CloudWatch

Corpus count: 62 (44 and 18 in the two directions).

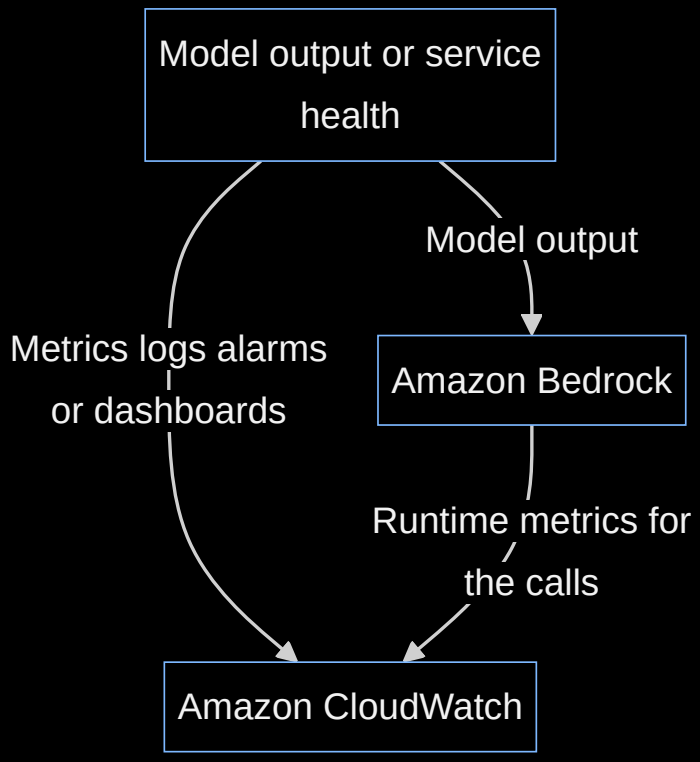


Figure: Bedrock produces the output. CloudWatch watches the calls.

- **Purpose:** Bedrock runs model inference. CloudWatch monitors resources and applications with metrics, logs, alarms, and dashboards.
- **Input:** Bedrock takes prompts. CloudWatch takes metric data points and log events from services.
- **Effect:** Bedrock returns model output. CloudWatch raises alarms and shows trends.

Ask yourself: Is the question asking the system to produce content, or asking how the system behaves over time?

- **Bedrock signal words:** generate, answer, draft, classify text, respond to users.
- **CloudWatch signal words:** alarm, threshold, dashboard, metric, log group, latency trend, error rate.

Trap: Bedrock publishes runtime metrics into CloudWatch, so the two services appear together. The monitoring answer is still CloudWatch. Bedrock does not become a monitoring service because its metrics appear there.

Source: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html, https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html, Sundog slide page 291.

Bedrock versus Step Functions

Corpus count: 53 (33 and 20 in the two directions).

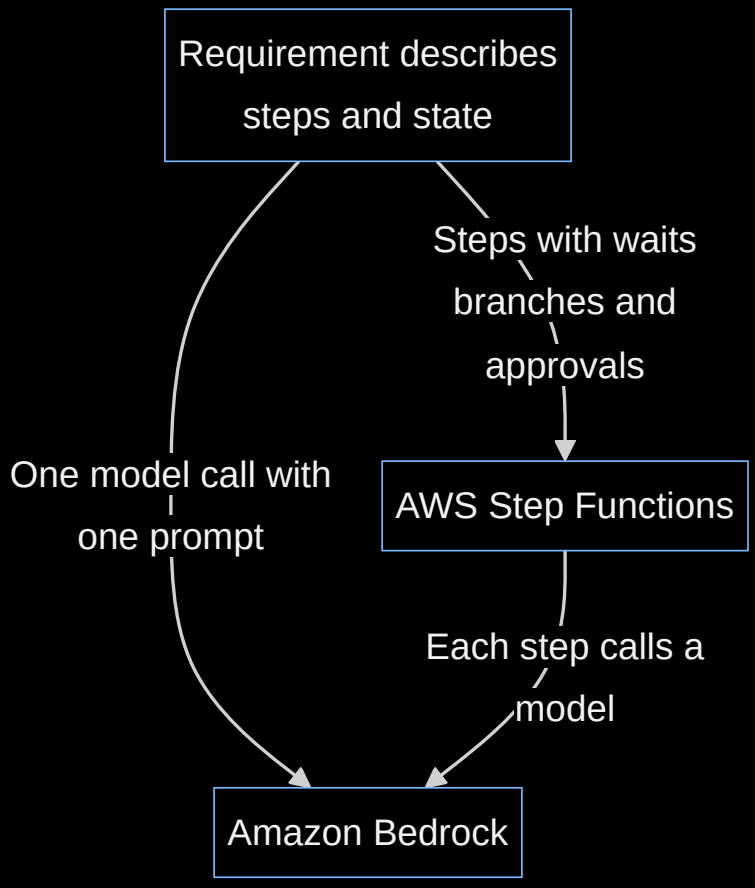


Figure: Step Functions owns the order of work. Bedrock owns one model call.

- **Purpose:** Step Functions coordinates a sequence of tasks with state, retries, waits, and parallel branches. Bedrock performs model inference.
- **Input:** Step Functions takes a state machine definition and a payload. Bedrock takes a prompt or messages.
- **Effect:** Step Functions records each execution and its state transitions. Bedrock returns one generated reply per call.

Ask yourself: Does the question care about what happens between steps, or about the text of one reply?

- **Step Functions signal words:** state machine, retry, wait, approval, branch, parallel, long-running, audit history.
- **Bedrock signal words:** single prompt, reply, model, token, summary, answer.

Trap: A state machine around one model call adds work with no benefit. When the stem asks only for a reply, Bedrock is the answer even if a workflow service is mentioned.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html, https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html.

Bedrock versus Guardrails

Corpus count: 44 (25 and 19, both with Bedrock as the correct answer, merged across both label forms).

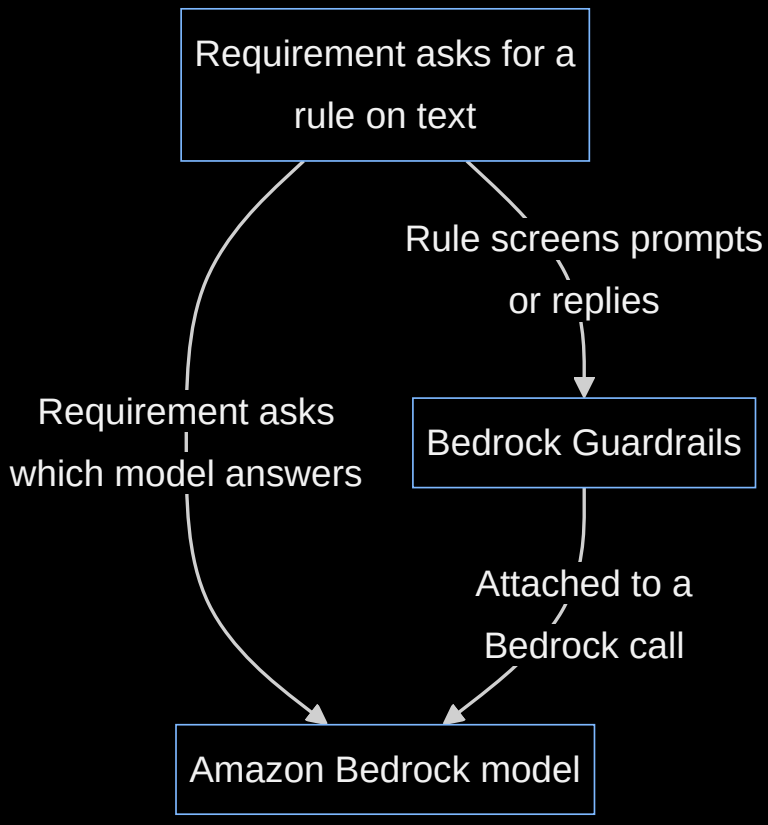


Figure: Guardrails is a policy layer. It does not pick or host a model.

- **Purpose:** Guardrails screens inputs and outputs against configured policies. Bedrock gives access to the model that produces the output.
- **Input:** Guardrails takes the text and a guardrail configuration. Bedrock takes the prompt and a model ID.
- **Effect:** Guardrails blocks, masks, or only detects content. Bedrock returns the model output, which Guardrails can then screen.

Ask yourself: Is the requirement a model choice or a policy that screens text in each request?

- **Guardrails signal words:** block, mask, denied topic, PII filter, harmful content, grounding threshold, policy.
- **Bedrock signal words:** model, invoke, converse, tokens, throughput, model ID.

Trap: Guardrails appears in the same answer as a model call, so it is easy to pick it as the model. A guardrail is attached to a request and does not return a reply by itself. The model still comes from Bedrock.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html, Sundog slide pages 21 and 57.

SageMaker endpoint versus Bedrock

Corpus count: 43 (38 with Bedrock as the correct answer, 5 with SageMaker as the correct answer).

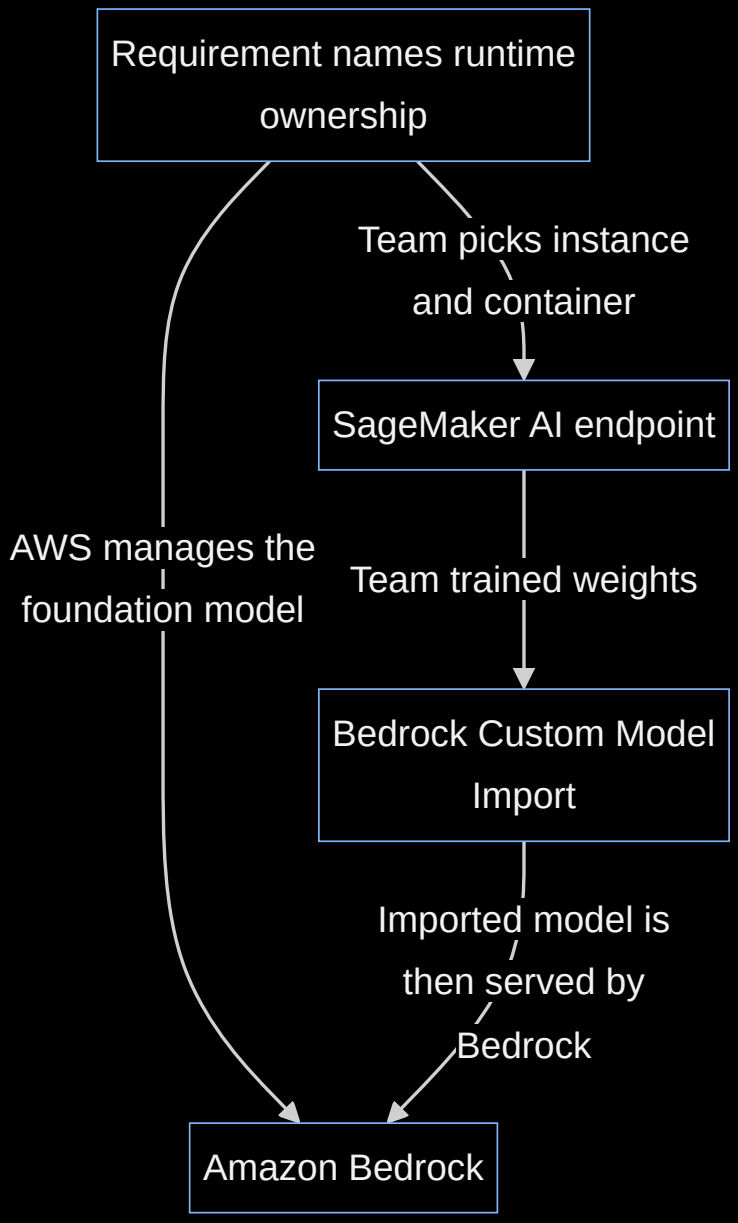


Figure: Custom weights can reach Bedrock, but the team still chooses the training path.

- **Purpose:** Bedrock gives API access to managed foundation models. A SageMaker AI endpoint hosts a model on infrastructure you choose.
- **Input:** Bedrock takes a model ID and a request body. SageMaker takes a model artifact, a container, and an instance type.

- **Effect:** Bedrock returns inference from a managed model. SageMaker returns inference from your hosted model, with control over scaling and deployment settings.

Ask yourself: Does the team need to own the serving stack, or does it need a managed model API?

- **Bedrock signal words:** managed foundation model, serverless inference, no instance choice, guardrails, provisioned capacity.
- **SageMaker signal words:** instance type, container, multi-model endpoint, asynchronous inference, model registry, own weights.

Trap: A model that was fine-tuned in SageMaker AI does not force a SageMaker endpoint. Bedrock can serve an imported model when the artifact and architecture are supported. A customized Bedrock model also needs Provisioned Throughput before you can call it.

Source: Sundog slide pages 291, 316, and 317. https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html.

Bedrock versus API Gateway

Corpus count: 39 (33 and 6 in the two directions).

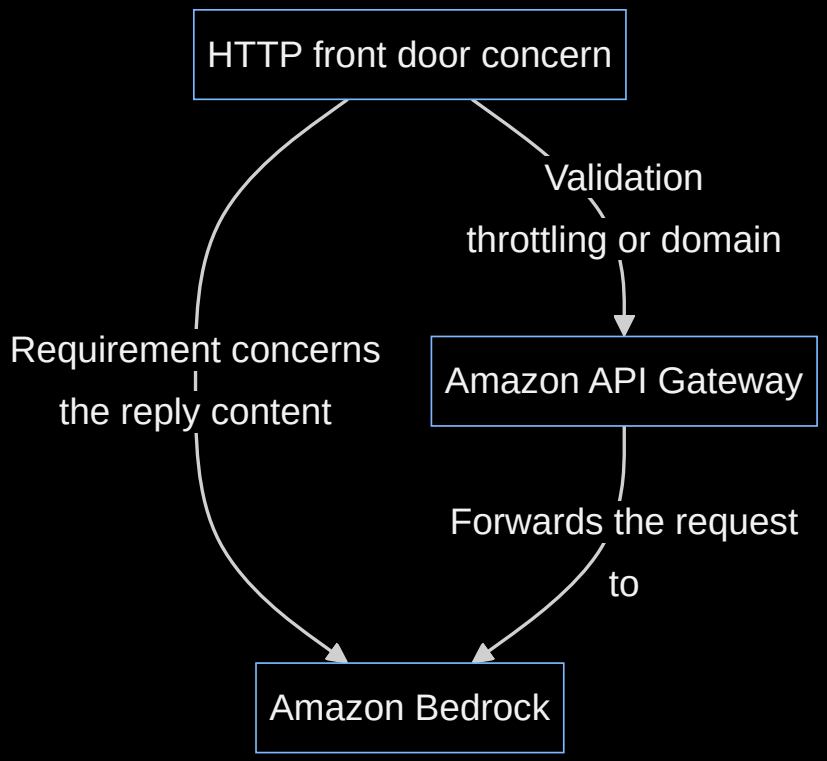


Figure: API Gateway receives and checks the call. Bedrock writes the reply.

- **Purpose:** API Gateway exposes an HTTP interface and applies request checks, throttling, and access controls. Bedrock produces model output.

- **Input:** API Gateway takes HTTP requests. Bedrock takes the payload that the backend sends.
- **Effect:** API Gateway accepts, rejects, or forwards a request. Bedrock returns generated content for an accepted request.

Ask yourself: Does the requirement control who calls the endpoint and what the request must contain, or does it control the reply?

- **API Gateway signal words:** request validation, usage plan, throttle, custom domain, WebSocket, WAF, routes.
- **Bedrock signal words:** model reply, summary, inference, token, generated text.

Trap: Adding API Gateway does not change the model. A stem that adds a front door to a model call still has Bedrock as the model service.

Source: https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html, Sundog slide pages 9 and 663.

Bedrock versus DynamoDB

Corpus count: 33 (26 and 7 in the two directions).

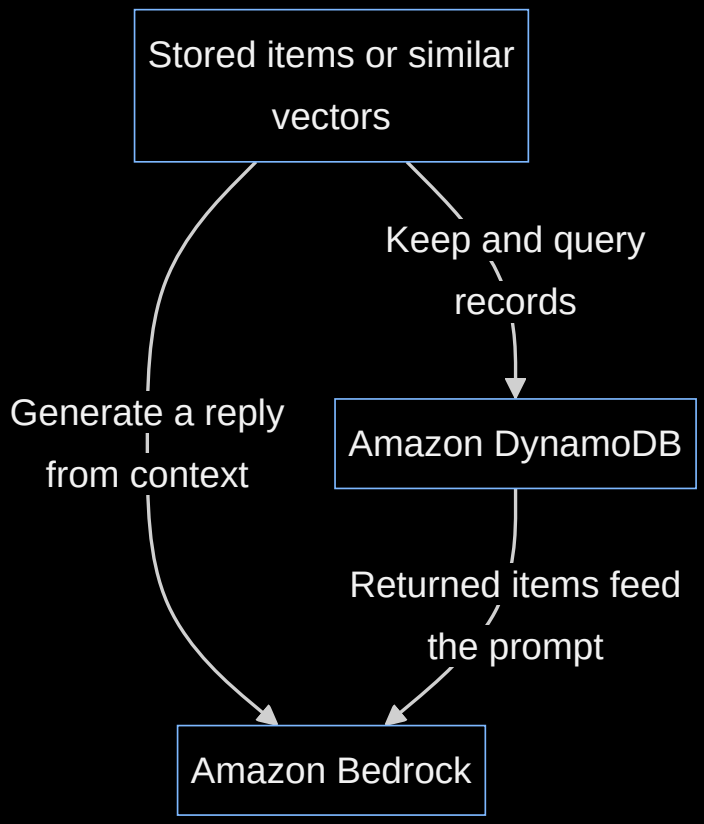


Figure: DynamoDB keeps and finds data. Bedrock writes the answer from it.

- **Purpose:** DynamoDB stores items and can run vector similarity searches with a vector index. Bedrock generates model output.
- **Input:** DynamoDB takes keys, items, and a query vector. Bedrock takes a prompt that may contain retrieved text.

- **Effect:** DynamoDB returns stored items and their similarity results. Bedrock returns a generated reply.

Ask yourself: Does the question need records returned, or a sentence written from records?

- **DynamoDB signal words:** items, keys, chat history, session store, vector index, similarity search, TTL.
- **Bedrock signal words:** generate, answer, summarize, explain, reply.

Trap: A DynamoDB vector index can return the nearest items for a query. It does not write a reply. The answer still needs a model call, which means Bedrock appears in the correct architecture.

Source: Sundog slide pages 186 and 187.

CloudWatch versus CloudTrail versus X-Ray

Corpus count: 33 (CloudWatch over CloudTrail 20, CloudTrail over CloudWatch 8, CloudWatch over X-Ray 5).

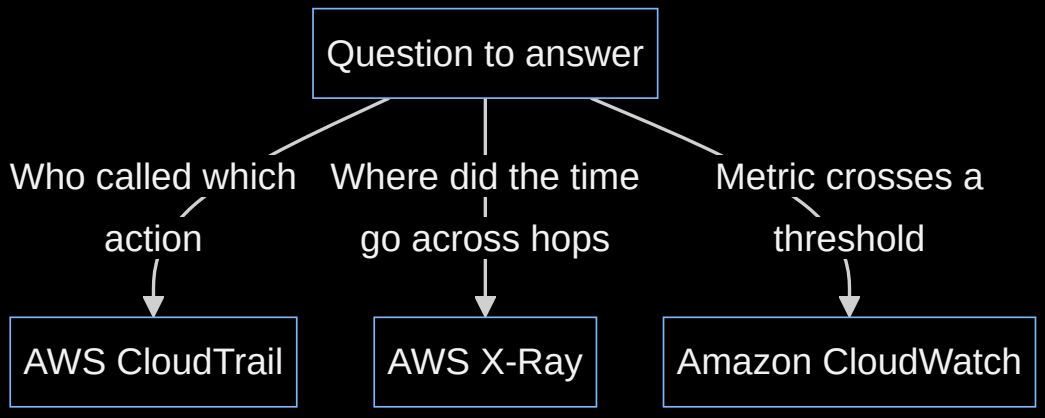


Figure: Each service answers one kind of question. Do not swap them.

- **Purpose:** CloudWatch monitors metrics, logs, and alarms. CloudTrail records account activity and API actions. X-Ray traces individual requests across services.
- **Input:** CloudWatch takes metric points and log events. CloudTrail takes API activity from the account. X-Ray takes trace segments from an SDK or from Lambda.
- **Effect:** CloudWatch raises alarms and shows dashboards. CloudTrail gives an audit record with the identity, time, and resource. X-Ray gives a trace map and timing for each hop.

Ask yourself: Is the question about who acted, which hop is slow, or which value crossed a line?

- **CloudWatch signal words:** alarm, threshold, dashboard, metric, log group, error rate.

- **CloudTrail signal words:** who, principal, audit, governance, management event, history.
- **X-Ray signal words:** trace, segment, service map, latency per hop, bottleneck, downstream call.

Trap: X-Ray shows where a request spent its time, but it does not tell you which user changed a setting. CloudTrail does the second job and does not show latency per hop.

Source: https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html, https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html, Sundog slide page 459.

Bedrock versus CloudTrail

Corpus count: 31 (27 and 4 in the two directions).

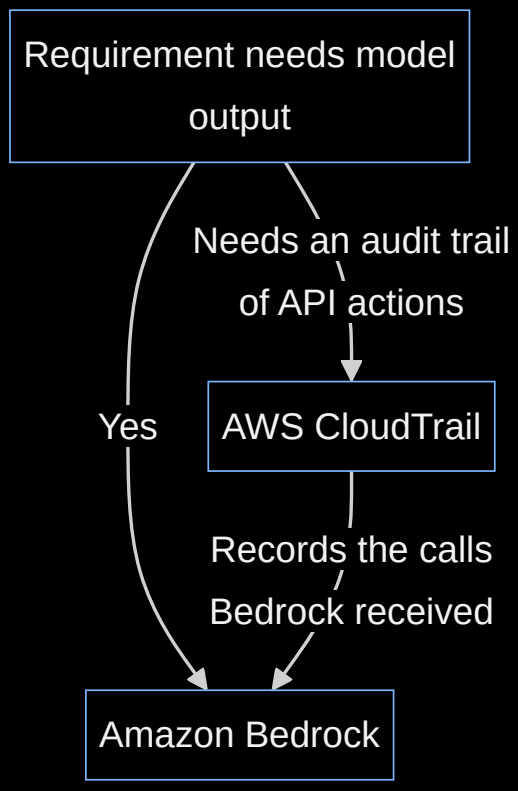


Figure: CloudTrail records that a call happened. It does not produce the reply.

- **Purpose:** Bedrock produces model output. CloudTrail records actions that users, roles, and services take, including API calls.
- **Input:** Bedrock takes prompts. CloudTrail takes the account API activity it records.
- **Effect:** Bedrock returns generated content. CloudTrail gives a searchable event record for audit.

Ask yourself: Does the stem need evidence that a call happened, or the content of the call?

- **Bedrock signal words:** generate, reply, token, model, guardrail, inference.

- **CloudTrail signal words:** audit, record, who invoked, compliance evidence, principal, event history.

Trap: CloudTrail is not a log of prompts and replies. A stem that asks for the text sent to a model needs a content log feature, not an API audit trail.

Source: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html, https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.html.

Lambda versus CloudWatch

Corpus count: 29 (16 and 13 in the two directions).



Figure: Lambda produces the signal. CloudWatch judges and alarms on it.

- **Purpose:** Lambda runs a function when an event arrives. CloudWatch stores metrics and logs and raises alarms when thresholds are crossed.
- **Input:** Lambda takes an event payload. CloudWatch takes metric data and log events.
- **Effect:** Lambda returns a result or performs an action. CloudWatch sends an alarm action or shows a dashboard.

Ask yourself: Is the requirement the code that acts, or the rule that decides when to act on a measured value?

- **Lambda signal words:** trigger, handler, event, transform, call, process.
- **CloudWatch signal words:** threshold, alarm, metric filter, dashboard, log insights, anomaly.

Trap: A Lambda function can check a metric and send a message. That is still monitoring logic, and the signal and the alarm belong to CloudWatch. Lambda is the action, not the monitor.

Source: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html, Sundog slide page 291.

Step Functions versus Lambda

Corpus count: 24 (12 and 12 in the two directions).

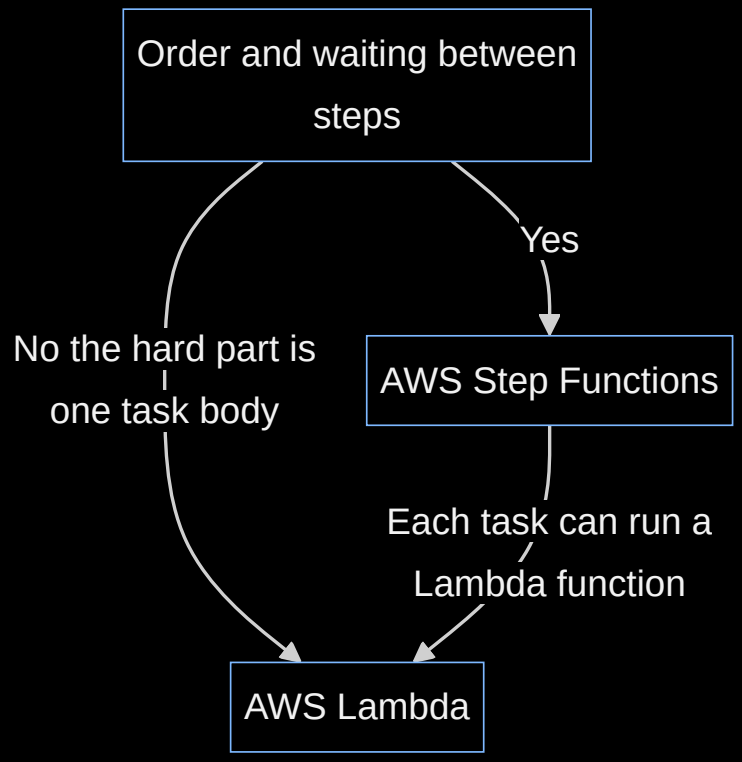


Figure: Step Functions manages the sequence. Lambda does one unit of work.

- **Purpose:** Step Functions runs a state machine with retries, waits, and branches. Lambda runs one function invocation.
- **Input:** Step Functions takes a state machine definition and an execution input. Lambda takes an event.
- **Effect:** Step Functions records state and execution history. Standard workflows run up to one year and follow exactly-once execution. Express workflows run up to five minutes and follow at-least-once or at-most-once execution.

Ask yourself: Does the stem care about the path through several steps, or about what one step computes?

- **Step Functions signal words:** wait, retry across steps, approval, branch, parallel, long-running, durable.
- **Lambda signal words:** single task, transform, call an API, handler, short run.

Trap: Retry loops and sleep logic written inside a Lambda function can work for short jobs. When the stem asks for durable state and an execution history, the orchestration belongs in Step Functions.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html.

Lambda versus EventBridge

Corpus count: 22 (17 and 5 in the two directions).

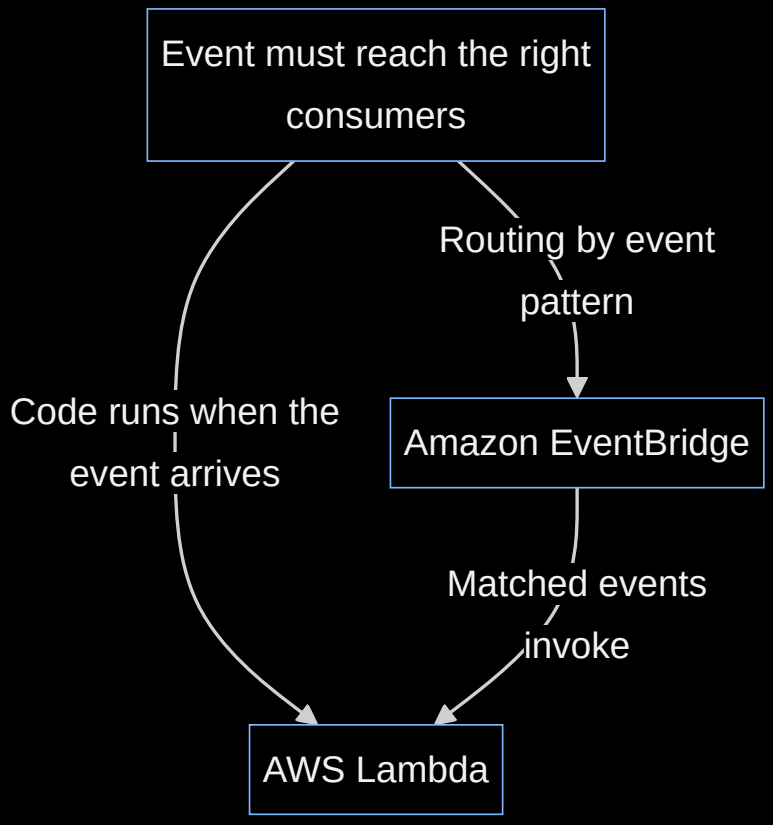


Figure: EventBridge decides who receives an event. Lambda does the work.

- **Purpose:** EventBridge routes events from sources to targets by rules. Lambda runs code for a single event.
- **Input:** EventBridge takes events from AWS services, applications, and partners. Lambda takes the events it is given.
- **Effect:** EventBridge delivers matched events to one or more targets. Lambda returns a result or performs an action.

Ask yourself: Does the stem ask which consumers receive an event, or what a consumer does with it?

- **EventBridge signal words:** event bus, rule, pattern, schedule, many targets, filter by attribute.
- **Lambda signal words:** process, compute, transform, handler, one consumer.

Trap: A single Lambda function that reads every incoming message and branches in code is not a router. When many sources must reach many targets by pattern, the routing belongs in EventBridge.

Source: https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html.

API Gateway versus Lambda function URL

Corpus count: 20 (all 20 in the direction where Lambda was the correct answer).

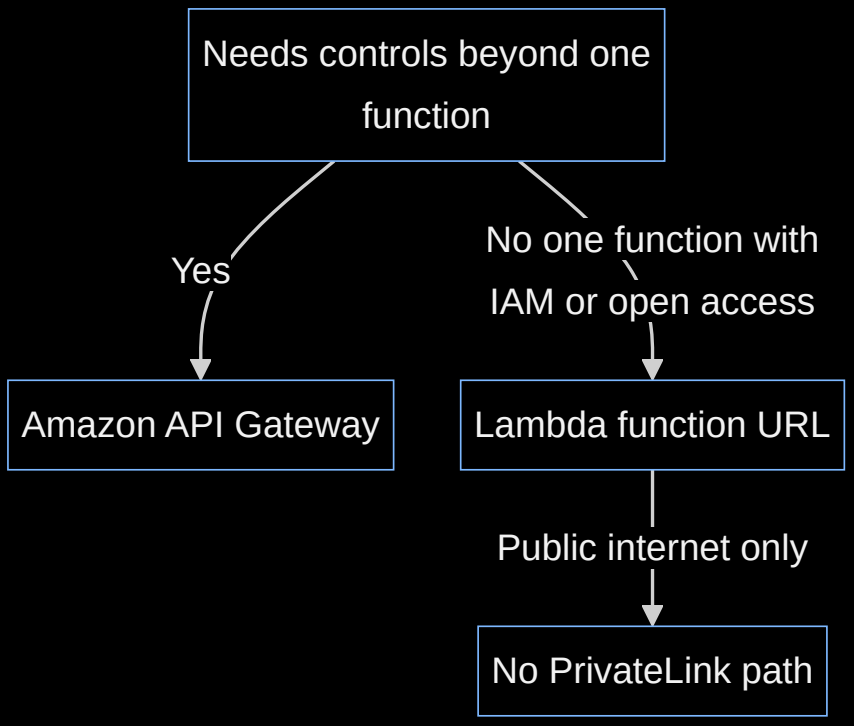


Figure: A function URL is simple. It is also public, so private access rules it out.

- **Purpose:** API Gateway is a full HTTP front door with routes and controls. A Lambda function URL is a dedicated HTTPS endpoint for one function or alias.
- **Input:** API Gateway takes requests on routes you define. A function URL takes HTTP calls for one function.
- **Effect:** API Gateway applies checks and forwards to a backend. A function URL invokes the function directly, with auth set to IAM or none, and CORS settings.

Ask yourself: Does the endpoint need controls beyond one function, such as validation across routes or a usage plan?

- **API Gateway signal words:** request validation, usage plan, API key, WAF, custom domain, many routes, WebSocket.
- **Function URL signal words:** one function, webhook, quick endpoint, CORS, IAM auth, reserved concurrency.

Trap: Function URLs use the public internet only. The page states that they do not support PrivateLink. A stem that requires private connectivity cannot pick a function

URL. The page also does not list request validation or usage plans, so Not in source for those features.

Source: https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html, Sundog slide page 663.

Bedrock versus Comprehend

Corpus count: 19 (15 and 4 in the two directions).

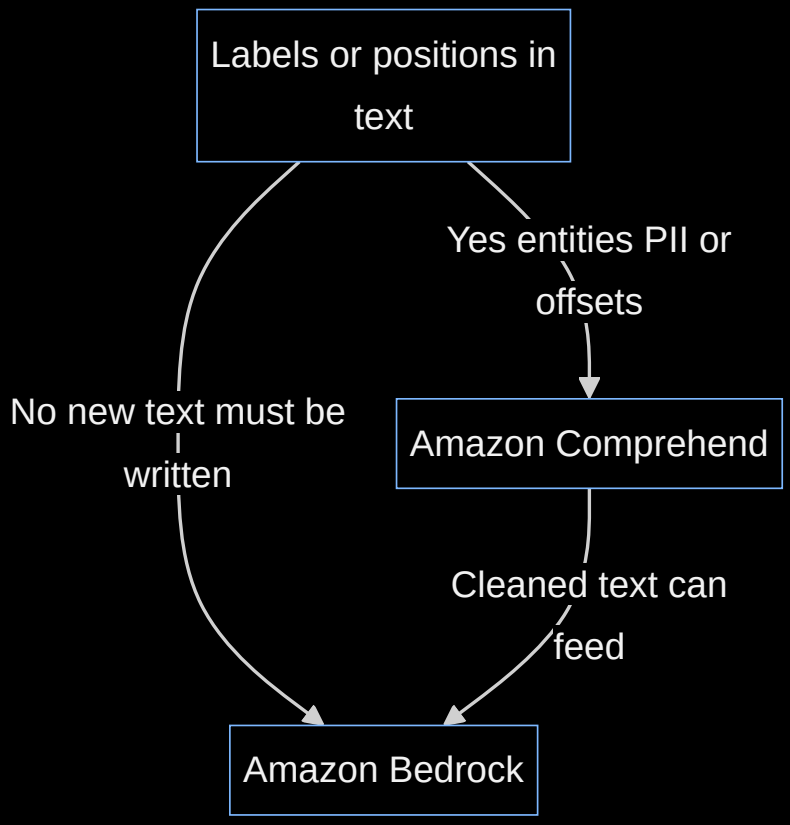


Figure: Comprehend reads and labels text. Bedrock writes text.

- **Purpose:** Comprehend analyzes text with natural language processing. Bedrock generates model output.
- **Input:** Comprehend takes UTF-8 text. For real-time PII detection, the page states up to 100 KB per document in English or Spanish. Bedrock takes prompts.
- **Effect:** Comprehend returns entity types, scores, and character offsets. Bedrock returns generated text.

Ask yourself: Does the output need a label and a location in the source text, or a new piece of writing?

- **Comprehend signal words:** detect, entities, PII, offsets, classify, sentiment, redact, language.
- **Bedrock signal words:** generate, summarize, draft, answer, reply.

Trap: Comprehend can find and redact PII before a model call. It does not decide the model reply or enforce a rule on the model output. When the stem asks for control of

the model reply, Guardrails or Bedrock is the answer.

Source: https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html, Sundog slide pages 57 and 118.

Guardrails versus Comprehend

Corpus count: 19 (11 with Guardrails as the correct answer and 8 with Comprehend as the correct answer, merged across label forms).

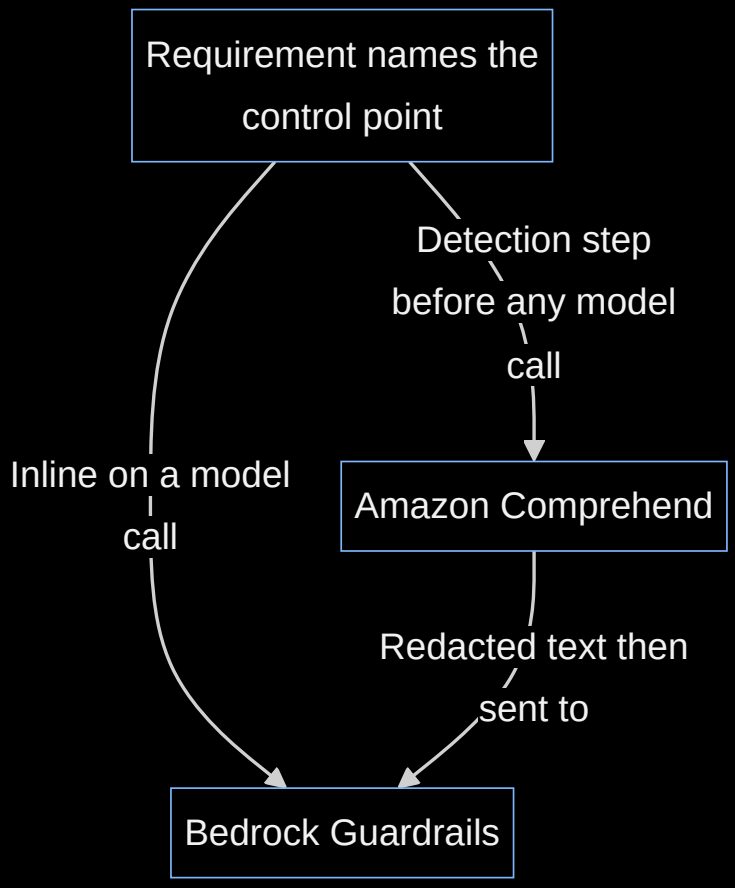


Figure: Comprehend can prepare the text. Guardrails enforces the policy on the model call.

- **Purpose:** Guardrails applies configured policies to prompts and model responses. Comprehend detects PII and other entities in text.
- **Input:** Guardrails takes the prompt and response text with a guardrail configuration. Comprehend takes text documents.
- **Effect:** Guardrails blocks, masks, or detects. Comprehend returns labels, offsets, or redacted copies. The caller decides what to do with the result.

Ask yourself: Must the control act on the model call itself, or is a separate detection step enough?

- **Guardrails signal words:** block, mask, denied topic, prompt attack, response, policy, inline.
- **Comprehend signal words:** detect, offsets, redact before inference, entity type, batch job.

Trap: Comprehend masking a field is not the same as a guardrail on the reply. If the stem asks that the reply itself never contain a blocked type, the guardrail is the control. A pre-step alone does not check the reply.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html, https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html, Sundog slide page 57.

Lambda versus SageMaker AI

Corpus count: 19 (9 and 10 in the two directions).

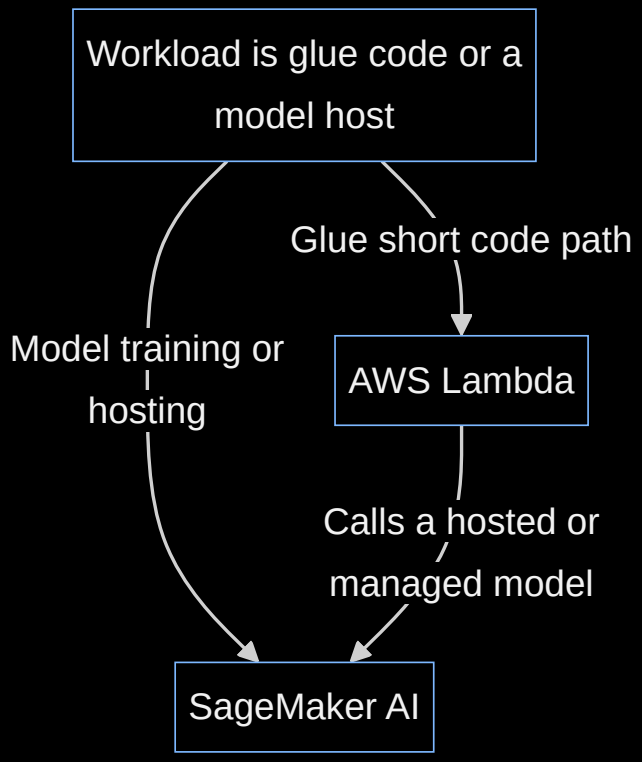


Figure: Lambda calls models. SageMaker AI is where you train or host your own.

- **Purpose:** Lambda runs short code in response to events. SageMaker AI builds, trains, and hosts machine learning models.
- **Input:** Lambda takes events. SageMaker takes training data, containers, and model artifacts.
- **Effect:** Lambda returns a result from code. SageMaker returns predictions from a deployed endpoint or batch job.

Ask yourself: Is the requirement to run code between services, or to own the model and its deployment?

- **Lambda signal words:** event, short, glue, call, transform, serverless function.
- **SageMaker signal words:** train, tune, endpoint, instance, model registry, asynchronous inference, inference component.

Trap: Lambda that calls Bedrock for a reply does not become a model host. When the team must choose the serving container or the instance, SageMaker AI is the answer

and Lambda is only the caller.

Source: Sundog slide pages 118, 316, and 317.

Provisioned Throughput versus batch versus on-demand

Corpus count: 17 for Bedrock as the on-demand answer over Provisioned Throughput. Batch has no direct row in the corpus.

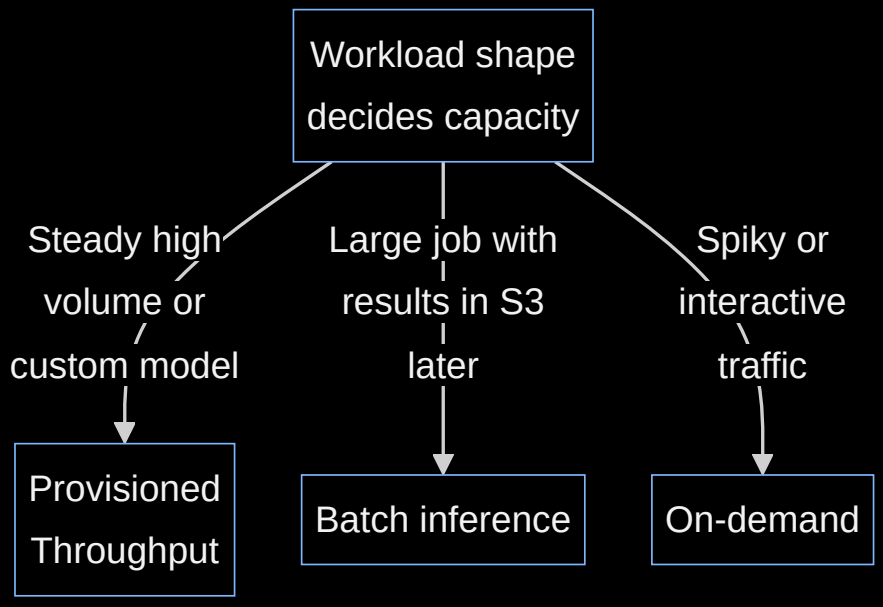


Figure: Pick the capacity model from the traffic shape, the timing of results, and the model type.

- **Purpose:** Provisioned Throughput reserves model units for a model at a fixed hourly price. Batch inference runs many prompts asynchronously. On-demand uses account quotas and pays per use.
- **Input:** Provisioned Throughput needs a purchased model ARN used as the model ID. Batch takes input files in S3. On-demand takes ordinary inference calls.
- **Effect:** Provisioned Throughput gives predictable capacity with a commitment choice of none, one month, or six months. Batch writes results to S3 when the job completes. On-demand returns each reply as it is generated, within quotas.

Ask yourself: Does the business need a reply now, a reply later, or a guaranteed rate every hour?

- **Provisioned Throughput signal words:** predictable, steady, model units, reserved, customized model, commitment.
- **Batch signal words:** overnight, large dataset, S3 input and output, no user waiting, asynchronous.
- **On-demand signal words:** spiky, interactive, sporadic, pay per use, account quotas.

Trap: Batch is not an answer for interactive chat, even at high volume. Provisioned Throughput is not an answer when traffic is sporadic and the stem asks for the lowest fixed cost. The page also states that batch inference is not supported for provisioned models.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html, https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html, Sundog slide pages 225, 290, and 291.

Bedrock versus Flows

Corpus count: 15 (10 and 5 in the two directions).

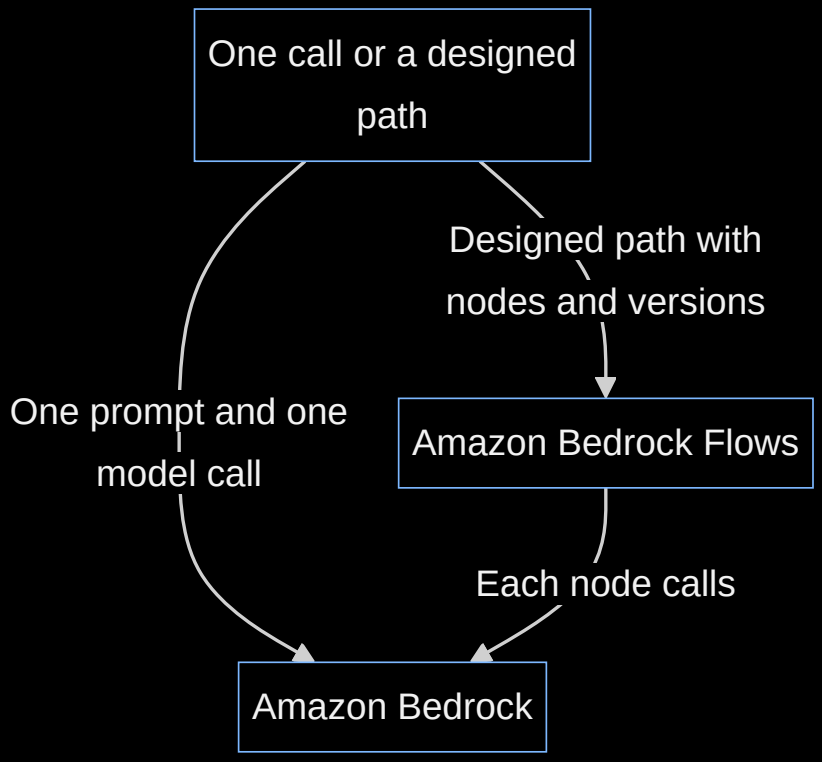


Figure: A flow is built from Bedrock calls. Use it when the path itself is the requirement.

- **Purpose:** Bedrock gives model access. Flows links prompts, models, knowledge bases, and AWS services into a visual workflow.
- **Input:** Bedrock takes a request. A flow takes an input and runs its nodes in order.
- **Effect:** Bedrock returns one reply. A flow returns the output of its final node. A flow can be versioned and deployed behind an alias.

Ask yourself: Is the requirement one model call, or a path that someone designs and changes over time?

- **Bedrock signal words:** single prompt, invoke, converse, one reply.
- **Flows signal words:** visual builder, nodes, versions, alias, invoke flow, prompt node, knowledge base node.

Trap: Flows is built on Bedrock, so Bedrock is in the answer either way. Choose Flows only when the stem asks for a designed path. A stem that asks only which model reads a document still points to Bedrock.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html.

Lambda versus Prompt Management

Corpus count: 15 (10 with Prompt Management correct over Lambda, 5 with Lambda correct over Prompt Management).

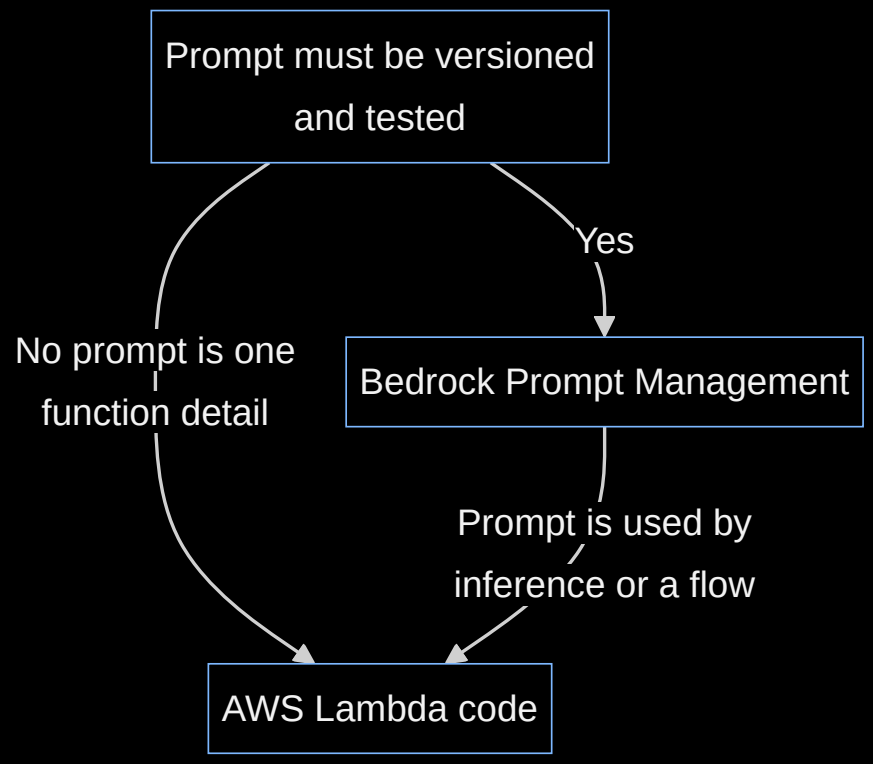


Figure: Prompt Management stores and versions the prompt. Lambda calls it.

- **Purpose:** Prompt Management stores prompt templates with variables, variants, and versions. Lambda runs code that can contain prompt strings.
- **Input:** Prompt Management takes prompt text, variables, and a model choice. Lambda takes events and calls.
- **Effect:** Prompt Management gives a versioned prompt that apps and flows can reference. Lambda returns the result of its code.

Ask yourself: Does the prompt need versions, variants, and reuse across several applications?

- **Prompt Management signal words:** reusable prompt, variant, version, variables, prompt builder, compare outputs.
- **Lambda signal words:** code, handler, one function, prompt in code, single app.

Trap: Keeping prompt text in Lambda code works for one app. It does not give the managed version and variant workflow that the stem asks for when several teams

reuse a prompt.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html, https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html.

CloudWatch versus EventBridge

Corpus count: 12 (CloudWatch over EventBridge, no reverse row).

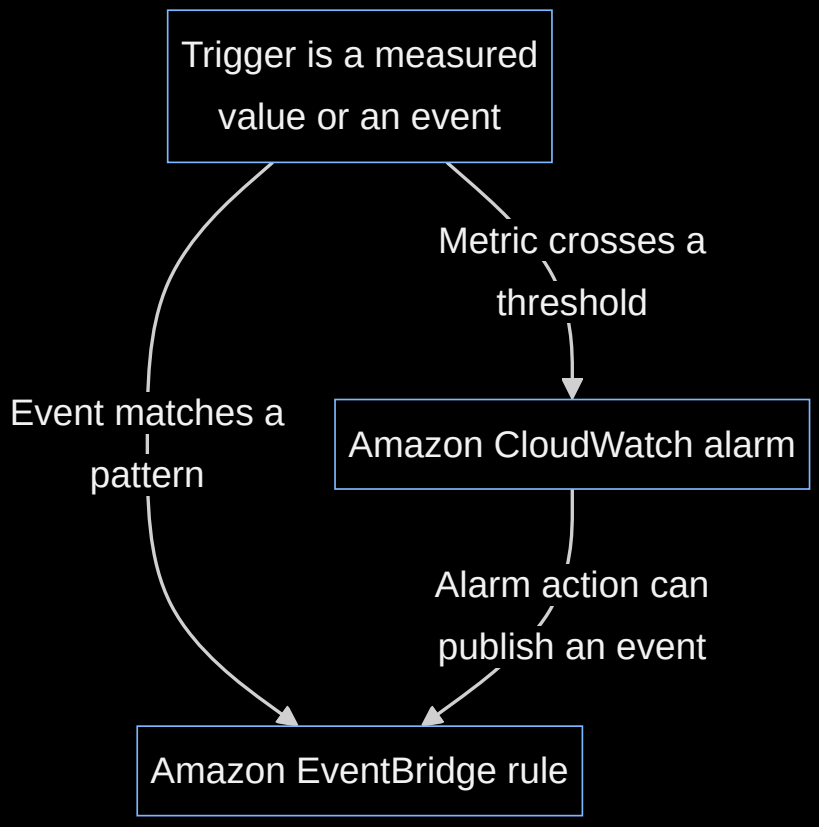


Figure: An alarm can emit an event, but the rule that matches events is EventBridge.

- **Purpose:** CloudWatch alarms watch metrics. EventBridge rules match events and route them to targets.
- **Input:** CloudWatch takes metric data. EventBridge takes events from services and applications.
- **Effect:** CloudWatch changes alarm state and can run an action. EventBridge delivers matched events to targets.

Ask yourself: Is the trigger a number that crossed a line, or an event whose attributes must be matched?

- **CloudWatch signal words:** threshold, metric, alarm, dashboard, anomaly.
- **EventBridge signal words:** event pattern, rule, target, schedule, route by attribute.

Trap: An alarm can trigger work, so it looks like routing. When the stem asks for routing of many event types by content, EventBridge is the answer.

Source: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html, https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html.

Flows versus Step Functions

Corpus count: 11 (Flows over Step Functions, no reverse row).

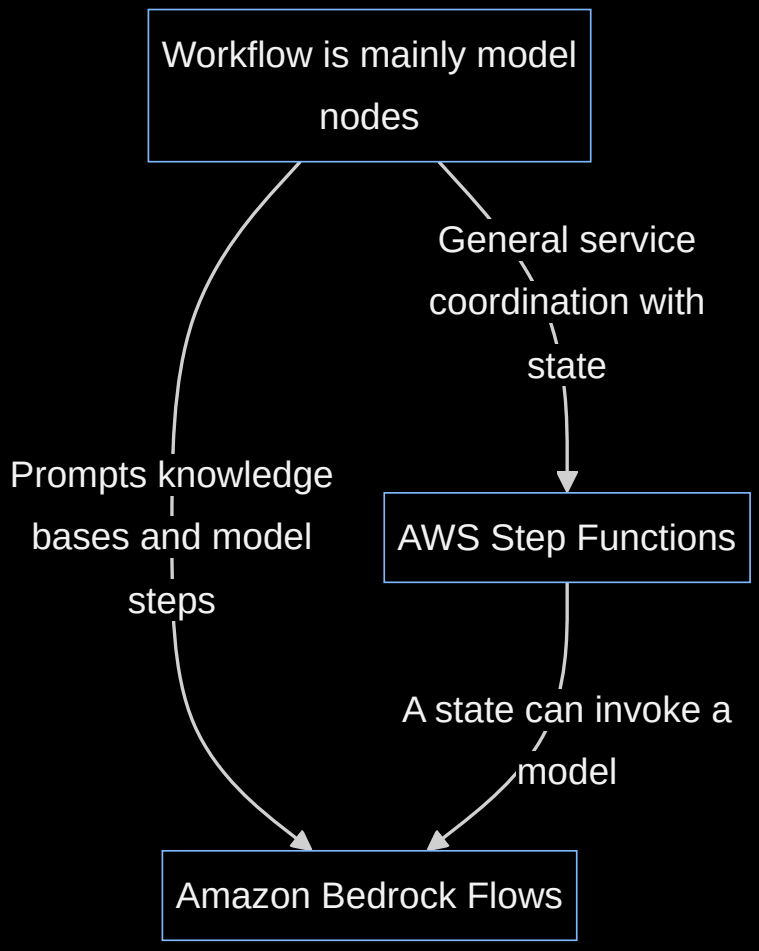


Figure: Flows is the model-first path builder. Step Functions is the general orchestrator.

- **Purpose:** Flows builds generative AI workflows from Bedrock nodes. Step Functions coordinates any set of AWS service tasks with state.
- **Input:** Flows takes a flow input and node configuration. Step Functions takes a state machine definition and an execution input.
- **Effect:** Flows returns the output of its nodes and can be versioned. Step Functions returns the final state and keeps an execution history.

Ask yourself: Are the steps mostly model and knowledge base nodes, or mostly service tasks with retries and waits?

- **Flows signal words:** prompt node, knowledge base node, analysts edit, visual builder, versions.

- **Step Functions signal words:** state machine, retry, wait, service integration, long-running, exactly once.

Trap: A flow is not a general orchestrator. If the stem needs durable waits, retries across many AWS services, or an execution history, Step Functions is the better fit even when a model step appears in the path.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html, https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html.

Amazon Q versus Bedrock

Corpus count: 11 (6 with Amazon Q correct over Bedrock, 5 with Bedrock correct over Amazon Q).

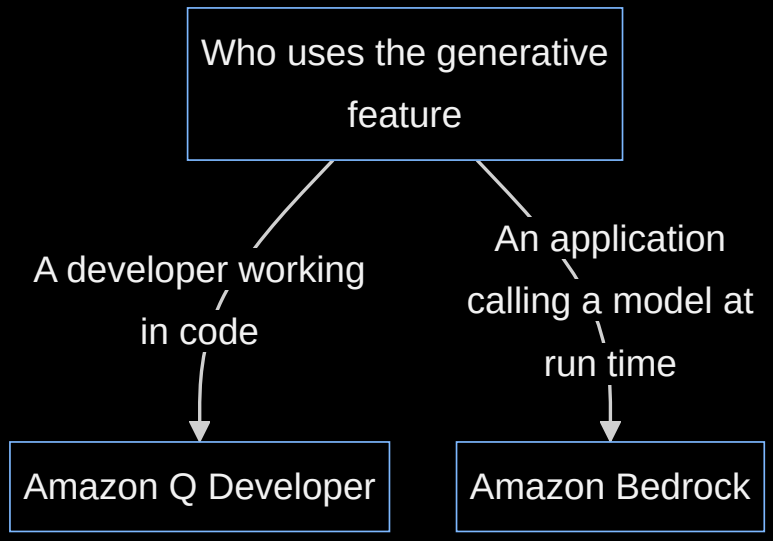


Figure: Q helps people build software. Bedrock is the API your software calls.

- **Purpose:** Amazon Q Developer assists developers with code work. Bedrock gives an application access to foundation models.
- **Input:** Q takes developer prompts and code context in a supported IDE. Bedrock takes API requests from your application.
- **Effect:** Q returns code help to the developer. Bedrock returns model output to your application.

Ask yourself: Is the person using the feature a developer working on code, or an end user of an application?

- **Amazon Q signal words:** IDE, developer productivity, code suggestions, refactor, tests for developers.
- **Bedrock signal words:** application, runtime, API call, end user, customers.

Trap: Amazon Q is not the service your application calls at run time. The Q product pages were not fetched for this module, so details beyond this split are Not in source.

Source: Sundog slide page 6 lists Amazon Q under agentic AI tools. Product detail for Amazon Q Developer: Not in source.

SQS versus EventBridge versus SNS

Corpus count: 5 (SQS over EventBridge). SNS has no direct row.

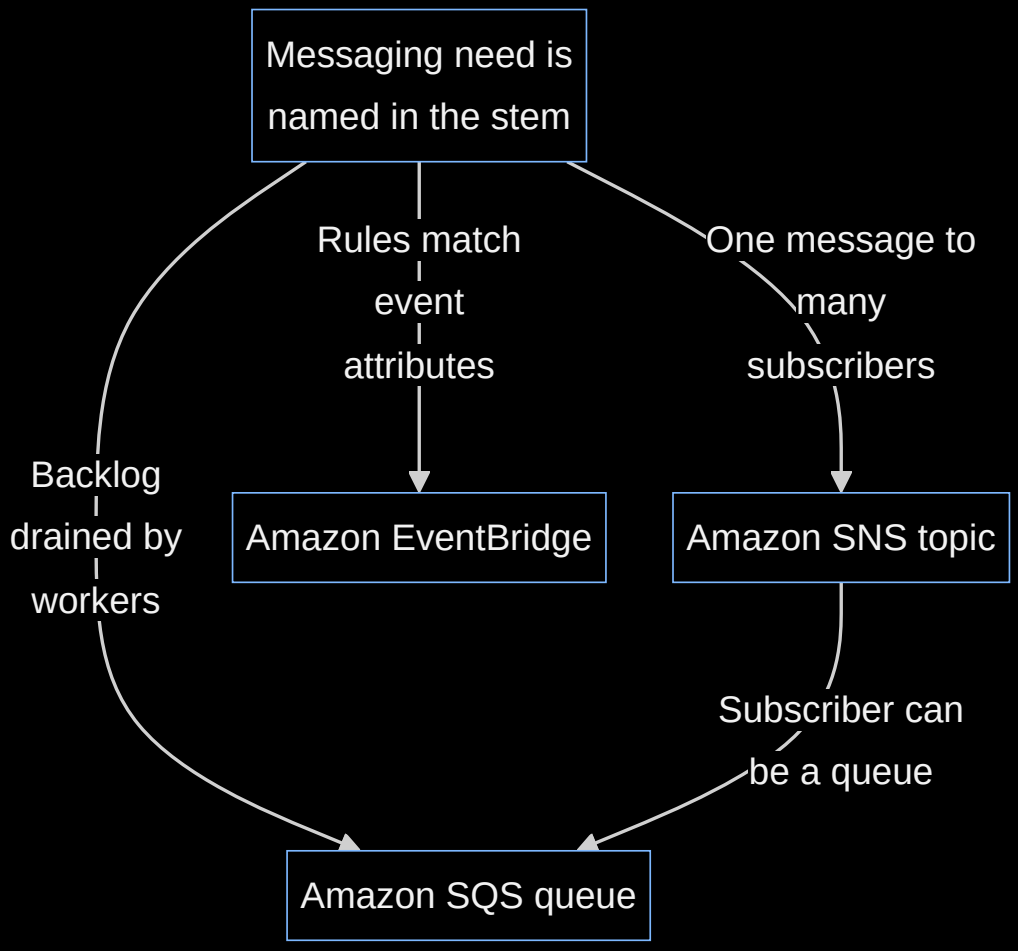


Figure: Queue for a backlog, topic for broadcast, bus for rule-based routing.

- **Purpose:** SQS holds messages until consumers receive and delete them. SNS publishes a message to a topic that fans out to many endpoints. EventBridge routes events by rules to targets.
- **Input:** SQS takes messages from producers. SNS takes published messages. EventBridge takes events from sources.
- **Effect:** SQS buffers work for consumers. SNS pushes copies to subscribers. EventBridge delivers each matched event to its rule targets.

Ask yourself: Is the message work to drain, a copy for each subscriber, or an event matched by its content?

- **SQS signal words:** queue, buffer, workers, decouple, retention, at least once, FIFO.

- **SNS signal words:** topic, publish, fan out, subscribers, email, push, many endpoints.
- **EventBridge signal words:** event bus, rule, event pattern, schedule, targets.

Trap: SNS feeding an SQS queue is a fan-out pattern, and both names appear in one correct design. Read the verb in the stem. A queue drains work, a topic broadcasts, and a bus filters by pattern.

Source: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html, https://docs.aws.amazon.com/sns/latest/dg/welcome.html, https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html.

OpenSearch versus Aurora pgvector versus S3 Vectors

Corpus count: 5 (OpenSearch over Aurora). S3 Vectors has no direct row.

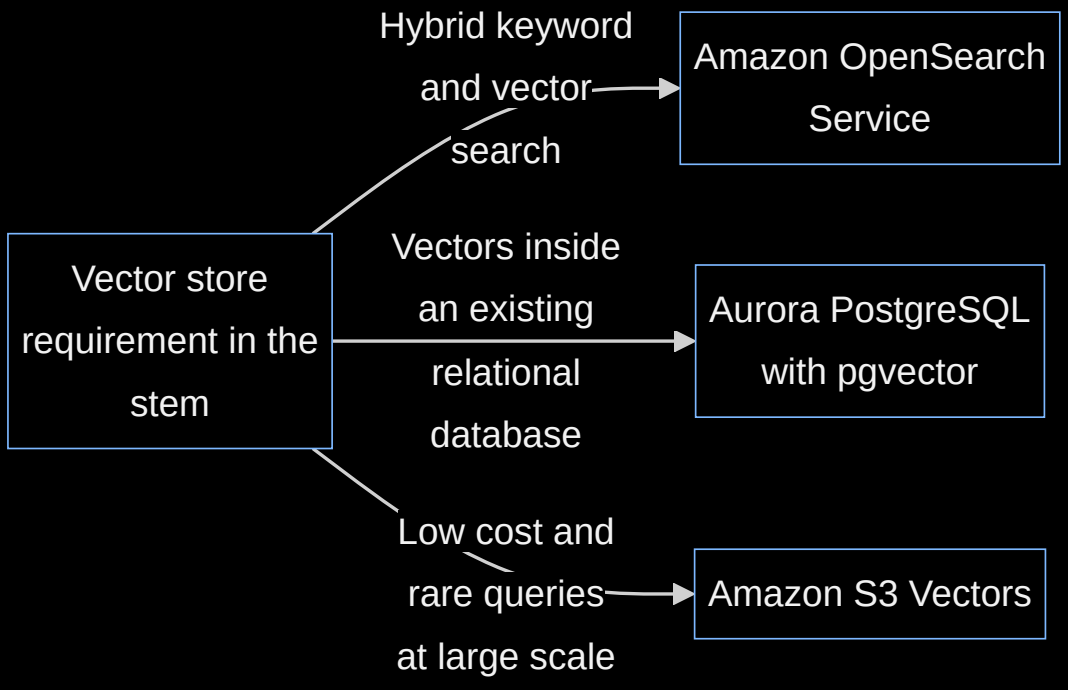


Figure: Pick the store from search style, existing database, and query rate.

- **Purpose:** OpenSearch offers search with keyword, vector, and hybrid queries. Aurora with pgvector keeps vectors next to relational data. S3 Vectors stores vectors in vector buckets and queries them by similarity.
- **Input:** OpenSearch takes indexed documents and vectors. Aurora takes rows with a vector column. S3 Vectors takes vectors and metadata in a vector index.
- **Effect:** OpenSearch returns ranked search hits. Aurora returns rows that match a vector and relational filters. S3 Vectors returns similar vectors with sub-second latency for infrequent queries.

Ask yourself: Does the query need keyword or hybrid search, or is it a similarity lookup where cost matters more than speed?

- **OpenSearch signal words:** hybrid search, keyword, full text, faceting, provisioned cluster.
- **Aurora pgvector signal words:** relational data, transactions, existing PostgreSQL, joins with vectors.
- **S3 Vectors signal words:** vector bucket, cost optimized, infrequent queries, metadata filter, storage at scale.

Trap: S3 Vectors is the cheapest option in the source, but the slides say it has higher latency than the database and search options. Limits for vector counts and metadata sizes: Not in source for this run.

Source: https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html, Sundog slide pages 139, 187, and 657.

Knowledge Bases versus Kendra

Corpus count: 0 direct. The nearest row is Bedrock over Kendra, with 6.

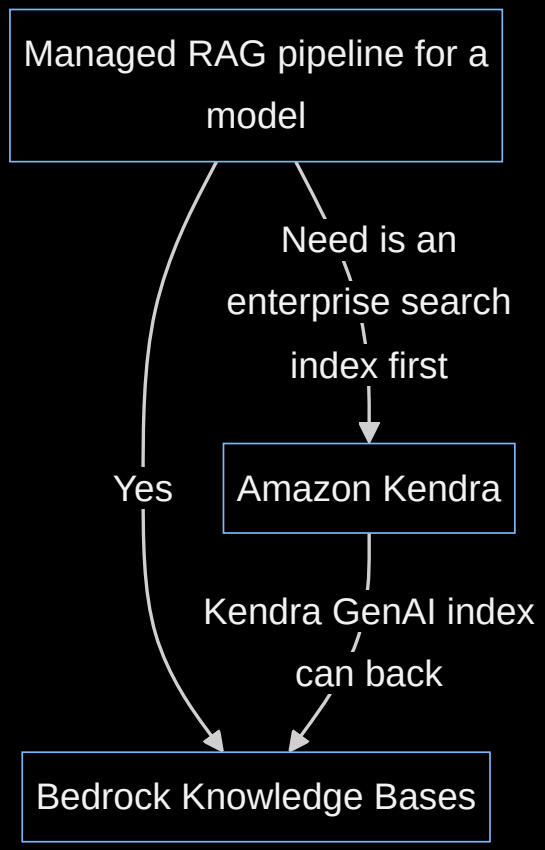


Figure: Kendra can be the search layer under a knowledge base. The two are not exclusive.

- **Purpose:** Knowledge Bases runs retrieval augmented generation with ingestion, chunking, embeddings, and a vector store. Kendra is an enterprise search service with an index and document access control.

- **Input:** Knowledge Bases takes documents from data sources. Kendra takes documents and connector content into an index.
- **Effect:** Knowledge Bases returns retrieved passages and can generate a cited answer. Kendra returns search results for users.

Ask yourself: Does the team need a search product with access control, or a managed retrieval layer that feeds a model?

- **Knowledge Bases signal words:** RAG, citations, embeddings, chunking, retrieve and generate, managed knowledge base.
- **Kendra signal words:** enterprise search, document-level access, index, connectors, search results.

Trap: Document-level permissions appear in both answers. The Knowledge Bases page says document-level permission filtering applies to managed knowledge bases, and that connectors and some features depend on the managed type. Check which type the stem describes.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html, Sundog slide pages 36, 657, and 662.

Model Evaluation versus CloudWatch

Corpus count: 5 (Model Evaluation over CloudWatch, no reverse row).

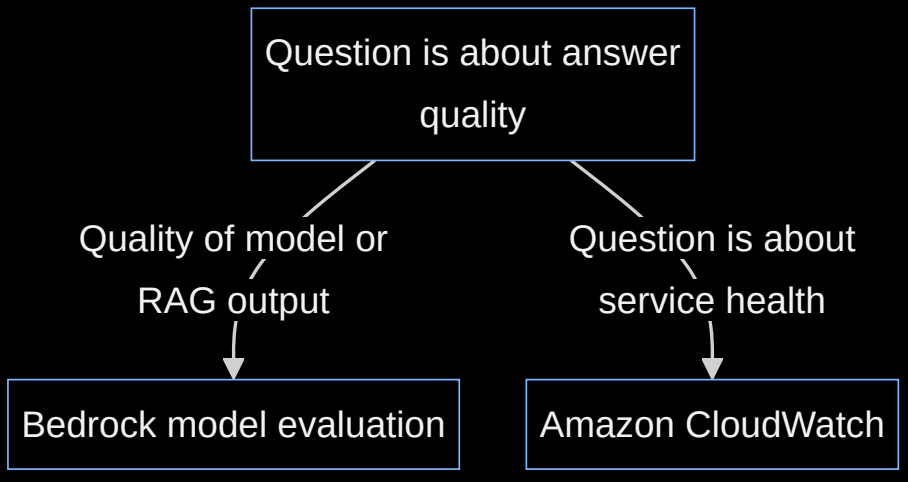


Figure: Evaluation grades the answers. CloudWatch watches the service.

- **Purpose:** Bedrock evaluations score a model or a knowledge base using automatic metrics, a judge model, or human workers. CloudWatch monitors operational health with metrics, logs, and alarms.
- **Input:** Evaluation takes a prompt dataset in S3 and reference answers when they are needed. CloudWatch takes metric data and logs.
- **Effect:** Evaluation returns quality scores and reports. CloudWatch returns alarms and dashboards.

Ask yourself: Does the stem measure whether the answers are correct, or whether the service is up and fast?

- **Model Evaluation signal words:** correctness, judge model, human rating, faithfulness, RAG retrieve only, retrieve and generate.
- **CloudWatch signal words:** latency, error rate, throttles, alarm, dashboard.

Trap: A dashboard of token counts and latency does not show answer quality. The quality question needs an evaluation job.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.html, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html, Sundog slide pages 430 and 431.

Guardrails sensitive information filter versus Macie

Corpus count: 0 direct. The nearest rows are Bedrock over Macie with 7 and Comprehend over Macie with 5.

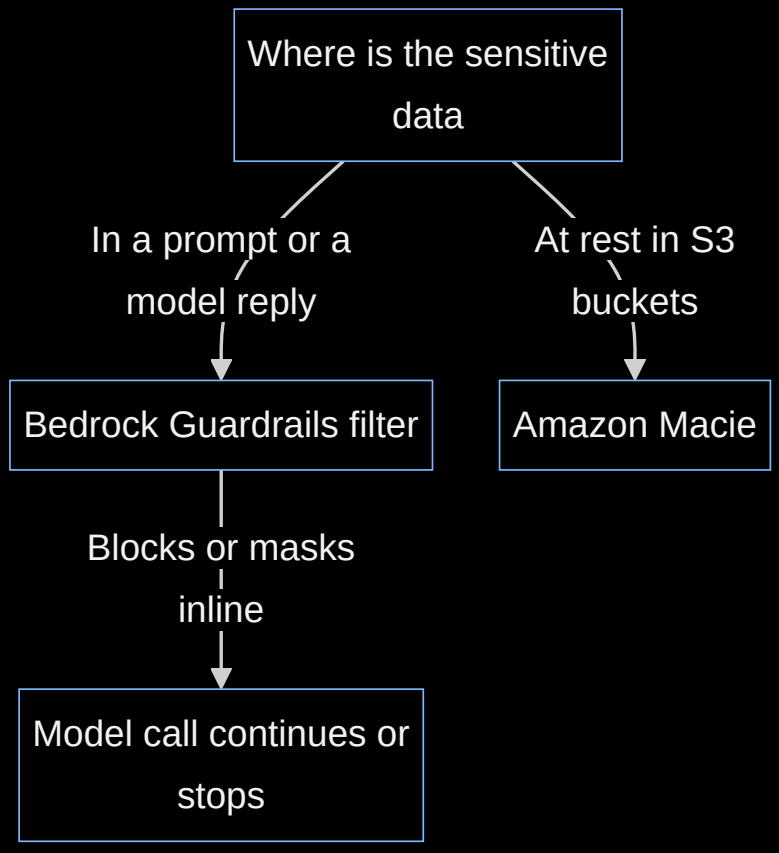


Figure: Data in motion to a model needs an inline filter. Data at rest needs discovery.

- **Purpose:** The sensitive information filter detects PII in prompts and responses and blocks or masks it. Macie discovers and classifies sensitive data in S3 and reports security findings.
- **Input:** The filter takes the text of a request or reply. Macie takes S3 objects and bucket inventory.

- **Effect:** The filter changes the text in the call. Macie produces findings for review and can publish them to EventBridge.

Ask yourself: Is the data moving to a model right now, or sitting in a bucket that needs an inventory?

- **Sensitive filter signal words:** prompt, response, mask, block, inline, per request, regex pattern.
- **Macie signal words:** S3 bucket, discover, classify, findings, at rest, inventory.

Trap: Macie does not filter a prompt before a model call. The guardrail filter does not inventory S3 buckets. The guardrail page also says masking does not change model invocation logs, so the logs need separate protection.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html, https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html.

Contextual grounding versus Automated Reasoning

Corpus count: 0 direct. Neither service has a corpus row in this set.

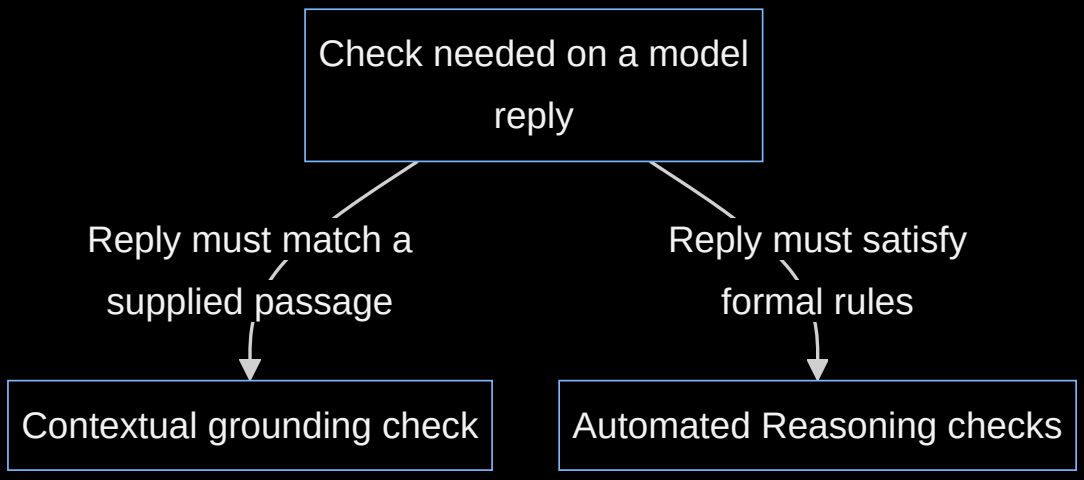


Figure: Grounding compares to a source passage. Automated Reasoning compares to a rule set.

- **Purpose:** Contextual grounding scores whether a reply is supported by a source and relevant to the query. Automated Reasoning checks a reply against a policy written as rules.
- **Input:** Grounding takes a grounding source, a query, and the reply. Automated Reasoning takes a policy built from source material.
- **Effect:** Grounding returns grounding and relevance scores, and blocks replies under the thresholds you set. Automated Reasoning returns the result of its rule checks.

Ask yourself: Is the proof a passage that supports the reply, or a rule that the reply must satisfy?

- **Grounding signal words:** source passage, grounded, relevant, hallucination, threshold, RAG answer.
- **Automated Reasoning signal words:** formal rules, policy, logical check, deterministic check.

Trap: A high grounding score does not prove that a rule is met. The grounding page covers only grounding and relevance. Details of the Automated Reasoning checks: Not in source for this run, because the fetched page did not render.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html. Automated Reasoning: Not in source.

Agents versus Flows

Corpus count: 0 direct. Bedrock over Agents has 10, and Flows appears only with Bedrock, Lambda, and Step Functions.

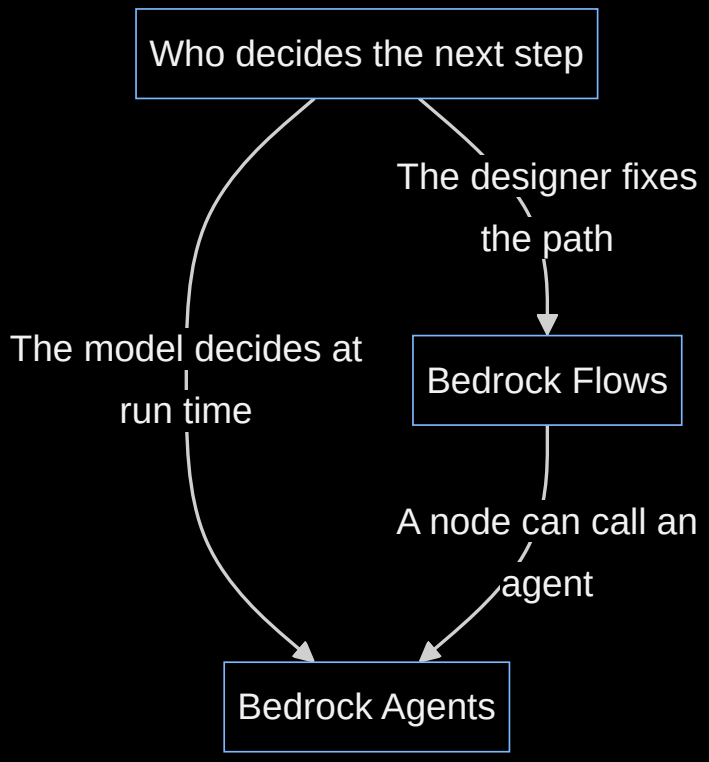


Figure: Agents plan with the model. Flows follows a path the designer draws.

- **Purpose:** An agent lets the model choose tools and knowledge sources for a task. A flow connects nodes in a path that the designer defines.
- **Input:** An agent takes a user request, action group definitions, and optional knowledge bases. A flow takes an input and the node configuration.
- **Effect:** An agent returns the result of the tools it chose. A flow returns the output of its nodes in the designed order.

Ask yourself: Should the model pick the next tool, or should the order be fixed in advance so auditors can read it?

- **Agents signal words:** decides, chooses tool, action group, plan, multi-step reasoning, varying requests.
- **Flows signal words:** fixed order, node, condition, analysts edit, deterministic path, version.

Trap: Picking Flows does not rule out agents. A flow can call an agent node. Read whether the stem fixes the order or leaves the order to the model.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html, Sundog slide pages 224 and 288.

Prompt Management versus Parameter Store

Corpus count: 0 direct. The nearest row is Bedrock over Parameter Store, with 7.

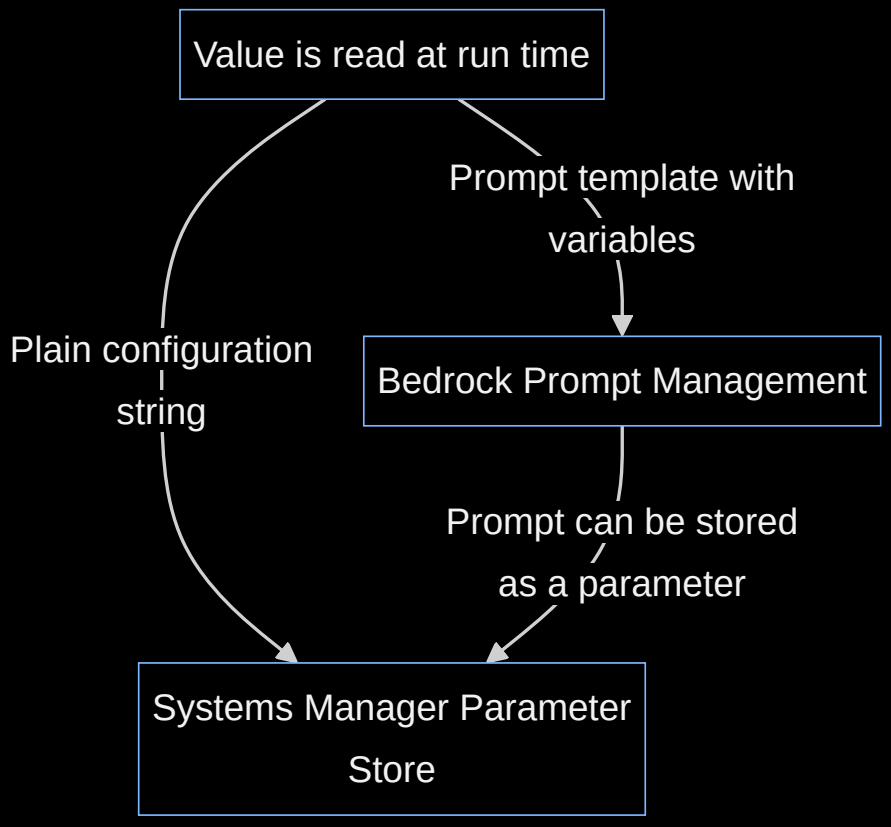


Figure: A prompt needs versions and variants. A configuration value needs a name and a value.

- **Purpose:** Prompt Management stores and tests prompt templates for models. Parameter Store stores named configuration values and secrets with versions.
- **Input:** Prompt Management takes prompt text, variables, and a model choice. Parameter Store takes a name and a value of type String, StringList, or SecureString.
- **Effect:** Prompt Management gives a prompt version that inference or flows can use. Parameter Store returns the value on each read.

Ask yourself: Is the stored value a prompt to compare and vary, or a setting such as an endpoint or a flag?

- **Prompt Management signal words:** prompt, variant, variables, model choice, compare outputs, prompt version.
- **Parameter Store signal words:** parameter, path, SecureString, endpoint, AMI ID, version history.

Trap: Parameter Store can hold prompt text, but it has no prompt builder, no variant comparison, and no model link. A stem that asks for prompt testing points to Prompt Management.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html, https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html.

Parameter Store versus Secrets Manager versus AppConfig

Corpus count: 0 direct. The nearest row is Bedrock over Parameter Store, with 7.

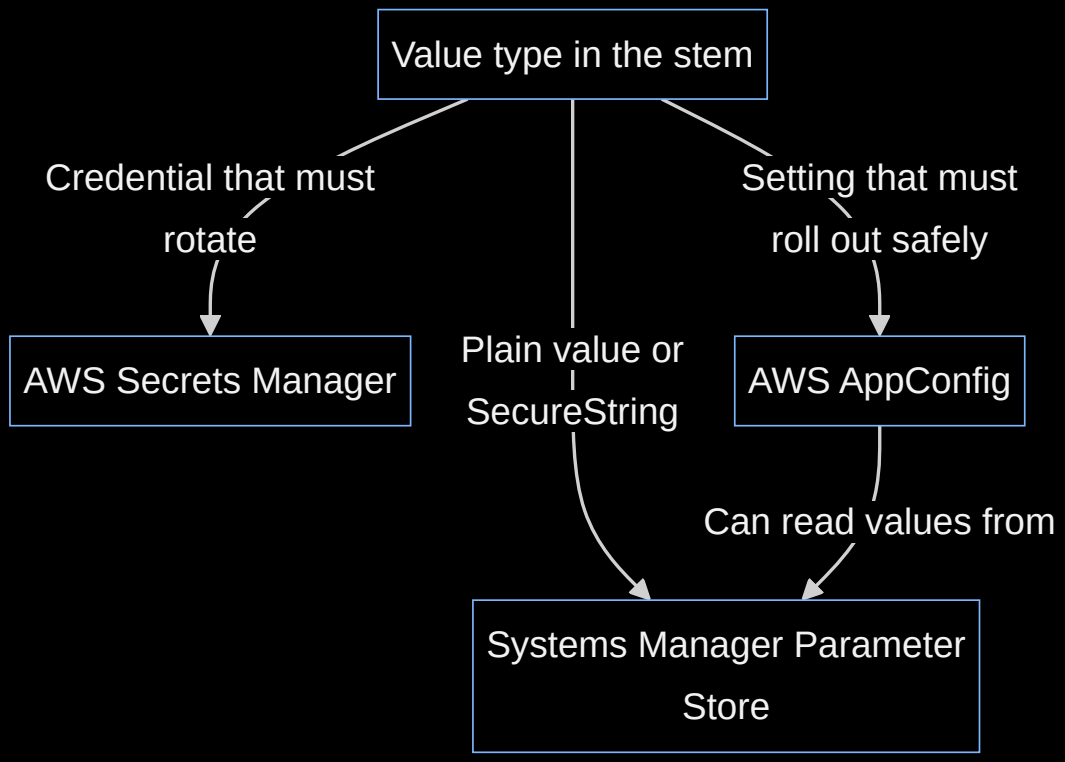


Figure: Secrets rotate, dynamic settings roll out with checks, and plain values stay simple.

- **Purpose:** Secrets Manager stores and rotates credentials. AppConfig deploys configuration and feature flags with validation and rollback. Parameter Store stores configuration values and SecureString values.
- **Input:** Secrets Manager takes a secret value and a rotation setting. AppConfig takes a configuration profile with validators. Parameter Store takes named values.
- **Effect:** Secrets Manager returns the current credential and can rotate it on a schedule. AppConfig rolls out the change gradually and rolls back on a CloudWatch alarm. Parameter Store returns the value on each read.

Ask yourself: Does the value need automatic rotation, a gradual rollout with rollback, or only a stored setting?

- **Secrets Manager signal words:** rotate, database credential, API key, OAuth token, rotation schedule.
- **AppConfig signal words:** feature flag, gradual rollout, validator, rollback, allow list, tuning.
- **Parameter Store signal words:** parameter, configuration value, version history, SecureString, hierarchy.

Trap: Parameter Store SecureString encrypts a value, but it does not rotate it. The Parameter Store page recommends Secrets Manager for credentials that need rotation. AppConfig can also read from Parameter Store, so the stem may name both.

Source: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html, https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html, https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html, Sundog slide page 364.

Fine-tuning versus RAG

Corpus count: 0. The pair does not appear in the corpus. It is kept because the learner confuses it often.

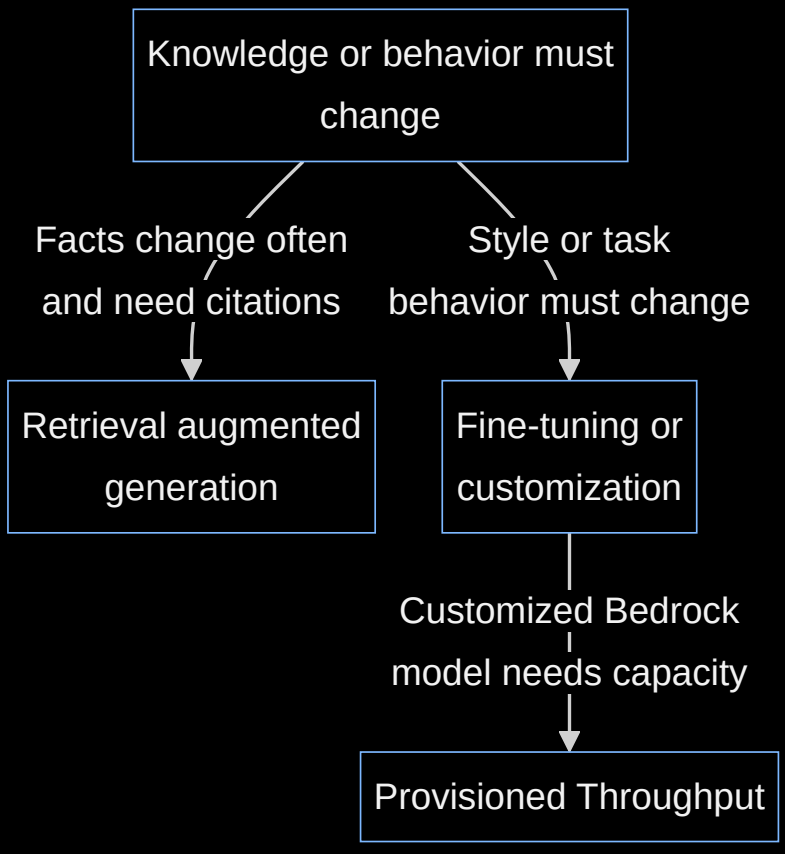


Figure: RAG adds passages at query time. Fine-tuning changes the model.

- **Purpose:** RAG retrieves passages from a source at query time and places them in the prompt. Fine-tuning trains the model on examples so that its behavior changes.
- **Input:** RAG takes a query and retrieves from documents. Fine-tuning takes labeled training examples and a base model.
- **Effect:** RAG answers from the current passages and can cite them. Fine-tuning changes the behavior of the model for future calls.

Ask yourself: Does the answer need up-to-date facts with sources, or a change in how the model writes and decides?

- **RAG signal words:** current documents, citations, source passage, updated often, knowledge base.
- **Fine-tuning signal words:** style, format, labeled examples, behavior, customized model.

Trap: Fine-tuning does not give citations to a source document. A customized Bedrock model also needs Provisioned Throughput before it can serve calls, so the cost and capacity choice follows the customization choice.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html, https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html, Sundog slide pages 36 and 317.

Read a long question

Each scenario below is short and original. Read the constraint words first, then remove each option by type. Name one reason for each removal. The answer comes last. Use the same method on the real stem.

Scenario one: policy answers with citations

A bank wants an assistant that answers loan questions from policy documents stored in S3. Each answer must show the paragraph that supports it. The team does not want to write retrieval code or run a vector database.

Constraint words: policy documents, show the supporting paragraph, no retrieval code, no vector database.

- A Lambda function with a self-managed vector store is out. It adds the retrieval work the stem bans.
- A SageMaker AI endpoint is out. It hosts a model, but it does not manage retrieval or citations.
- Comprehend is out. It labels text, but it does not write a cited answer.

Answer: Bedrock Knowledge Bases with a managed knowledge base. It runs retrieval and generation and returns the passages used.

Scenario two: data screened in each request

A clinic sends free text from patients to a model. Patient names and phone numbers must be masked before the model reads the text. Staff also want the model to refuse dosage advice, and that rule must act on each request.

Constraint words: masked before the model reads, refuse a topic, each request.

- Macie is out. It scans S3 objects at rest, so it does not screen a live prompt.
- A regex in Lambda alone is out. Regex misses names that depend on context, and it cannot enforce a topic rule.
- Comprehend alone is out. It masks PII, but it does not enforce a topic rule on the model reply.

Answer: Bedrock Guardrails with a sensitive information filter set to mask and a denied topic. The guardrail acts inline on the prompt and the reply.

Scenario three: who acted and how slow it was

An operations team must find which IAM principal changed a model access setting last week. The same team needs an alert when the latency of the assistant stays above a threshold for ten minutes. The two answers must come from different evidence.

Constraint words: which principal, last week, alert, threshold, latency.

- X-Ray is out for the first need. It traces requests, and it does not record who changed a setting.
- CloudTrail alone is out for the second need. It records actions, and it does not raise a latency alarm.
- Model Evaluation is out for both. It scores answer quality, and it does not record actions or watch latency.

Answer: CloudTrail for the principal and change record, and a CloudWatch alarm on latency. Each service answers only its own question.

Scenario four: a large job with results later

A publisher must summarize two million archived articles before the next morning. Results go to an S3 bucket, and no user waits for a reply. The team wants the lowest cost for this job.

Constraint words: two million, before morning, results in S3, nobody waits.

- A Lambda loop calling on-demand inference is out. It is synchronous, and the stem expects a batch process with results in S3.
- Provisioned Throughput is out. The batch page states that batch inference does not support provisioned models, and a commitment is not the lowest cost for a one-time job.
- Step Functions express synchronous execution is out. It has a five-minute limit, and the job is too large for one execution.

Answer: Bedrock batch inference with input and output in S3.

Scenario five: analysts change the path

Business analysts must change the steps of a generative workflow without writing code. Each step calls a prompt, a knowledge base, or a Lambda function. The team wants versions of the workflow and an alias that the application calls.

Constraint words: analysts without code, prompt, knowledge base, versions, alias.

- Agents are out. The model would choose the tools, and the stem fixes the path for analysts.
- Lambda alone is out. Each change would need a code deployment.
- Step Functions is out. It coordinates service tasks in a state machine, and the stem centers on prompt and knowledge base nodes that analysts edit.

Answer: Bedrock Flows. The visual builder, versions, and alias match the stem.

Exam tip: Write the constraint in five words or fewer before you read the options. Then remove options by type first. A type removal takes one sentence. A detailed comparison of two near-identical options is slower and less reliable.

Distractor patterns seen in the tests

Most wrong options are not random. They follow a small set of patterns. Recognizing the pattern lets you reject an option without knowing every detail of the service.

- **Right family, wrong layer.** A policy feature is offered as a model choice, or a model is offered as a control. It is wrong because each layer has a different job, and a policy does not pick a model.
- **Right purpose, wrong time.** A tool that checks data after it sits in storage is offered for data in a live request. It is wrong because the stem asks for a control at request time.

- **Right service, wrong network surface.** A private endpoint for one API set is offered for another API set. It is wrong because each Bedrock API surface has its own endpoint.
- **Build it yourself.** Custom code is offered where a managed feature exists. It is wrong when the stem asks for less operational work or for a managed control.
- **Managed feature where ownership is required.** A managed model API is offered when the stem asks for control of the serving container. It is wrong because the stem names the ownership boundary.
- **Wrong capacity model.** Committed capacity is offered for sporadic traffic, or on-demand is offered for a steady high load with a fixed rate need. It is wrong because the capacity choice follows the traffic shape.
- **Storage posed as retrieval or retrieval posed as storage.** A store that returns items is offered as an answer writer, or a model is offered as a store. It is wrong because the stem asks for one of the two jobs.
- **Monitoring posed as evaluation.** A dashboard is offered for answer quality. It is wrong because health metrics do not show whether an answer is correct.
- **Config posed as secret.** A plain parameter store is offered for a credential that must rotate. It is wrong because rotation belongs to the secrets service.
- **Heavy orchestration for a simple call.** A workflow service is offered for one model call. It is wrong because it adds state and cost with no requirement that needs it.

Drill

Read each prompt, then cover the answer line. Say the service and one reason before you read on.

1. A stem asks to mask names in each prompt before the model reads it, with the rule applied inline.

   Rule: Bedrock Guardrails sensitive information filter on the model call, set to mask.

2. A stem asks which principal changed a model access setting last month.

   Rule: CloudTrail event history or a trail.

3. A stem asks where the time goes across three services in one slow request.

   Rule: X-Ray trace timeline and service map.

4. Analysts must change the order of prompt and knowledge base steps without code.

   Rule: Bedrock Flows with versions and an alias.

5. A workflow needs retries, waits, and a manual approval step with a durable history.

   Rule: Step Functions standard workflow.

6. Ten thousand documents must be summarized by morning, with results written to S3.

   Rule: Bedrock batch inference with S3 input and output.

7. A custom model must serve steady traffic with predictable capacity every hour.

   Rule: Provisioned Throughput with model units.

8. A stem asks to find PII in documents already stored in S3 buckets.

   Trap: Macie. It discovers sensitive data at rest, and it does not filter live prompts.

9. A stem asks for character offsets of PII in one text string.

   Rule: Amazon Comprehend PII detection on text.

10. A reply must be checked against a supplied source passage and a user query.

    Rule: Contextual grounding check. Set a threshold for grounding and a threshold for relevance.

11. A reply must satisfy rules written as formal logic from policy documents.

    Rule: Automated Reasoning checks. Details of the feature: Not in source for this run.

12. A webhook must reach one function over private connectivity from a VPC.

    Trap: A function URL is public internet only. It does not use PrivateLink, so pick a private path instead.

13. An endpoint needs request validation, usage plans, and WAF across several routes.

    Rule: Amazon API Gateway.

14. Workers must drain a backlog of messages at their own pace.

    Rule: Amazon SQS queue.

15. One event must reach several subscribers at the same moment.

    Rule: Amazon SNS topic with subscribers.

16. Events must be matched by content and sent to different targets, with scheduled runs.

Rule: Amazon EventBridge rules.

17. A prompt template needs versions, variables, and variant comparison before release.

    Rule: Bedrock Prompt Management.

18. A configuration change must roll out gradually and roll back on a CloudWatch alarm.

    Rule: AWS AppConfig with a deployment strategy.

Sources

AWS documentation pages fetched for this module:

- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html
- https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html
- https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html
- https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html
- https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html
- https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html
- https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html
- https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html
- https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html

- https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html
- https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html
- https://docs.aws.amazon.com/sns/latest/dg/welcome.html
- https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome
- https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html

Course slides used as a second source: Sundog slide pages 6, 9, 21, 36, 57, 118, 139, 186, 187, 224, 225, 288, 290, 291, 316, 317, 364, 430, 431, 459, 657, 662, and 663.

Not in source for this run: Automated Reasoning checks feature details, Amazon Q Developer product details, Lambda function URL features such as usage plans, and S3 Vectors numeric limits.

All modules Previous